



Agents et frameworks agentiques

CEPE mars 2026

Alexandre Tuel, Thomas Kerdreux & Antoine Audras

tkerdreux@galeio.fr



**INTRODUCTION
AUX AGENTS**

1

**CONSTRUIRE UN
AGENT**

2

**MÉMOIRE ET
RAG**

3

STRUCTURE DU COURS

4

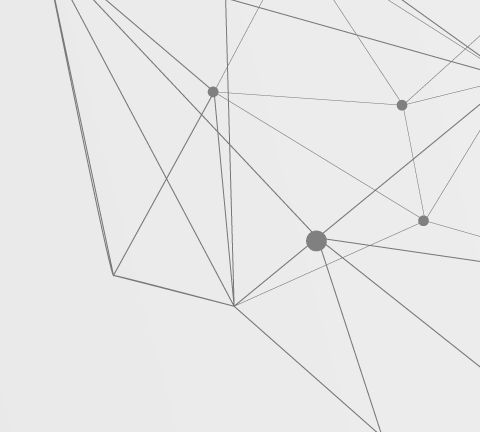
**AGENTS FIABLES :
GRAPHES ET ÉTATS
STANDARDS**

5

**OBSERVABILITÉ ET
SÉCURITÉ**

6

**CONCLUSION/
ALLER PLUS LOIN**





LangGraph



smolagents



AutoGen



Agents

MCP

Observability

*Function
calling*

Memory

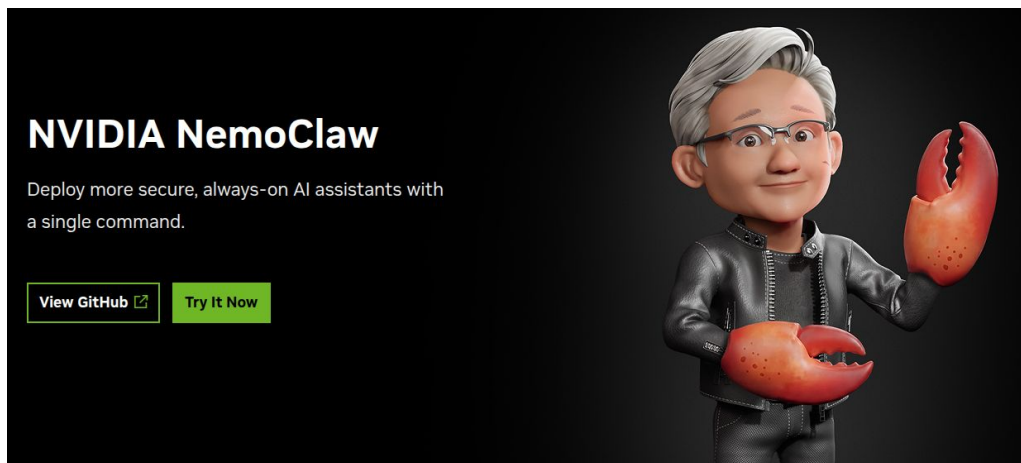
Quelques commentaires pour commencer

- Sujet **très récent**, encore en **évolution rapide** (de nouvelles choses chaque semaine...)
- **Très peu d'IA**, surtout des enjeux de software engineering : on va peu parler des LLMs, mais plutôt d'architectures de code, de RAG, de tools, de mémoire, de prompts, d'instructions, etc.
- Très peu de théorie unifiée et stabilisée aussi : tout est dans la pratique. Il n'y a pas de théorie sur ce que doit être un bon agent ou comment l'implémenter. Juste du trial-and-error. Tout reste encore confus et brouillon.
- Au fond rien de très compliqué dans les agents : il faut juste se faire une mental map claire de ce dont on a besoin
- On ne va pas parler des systèmes multi-agents mais au fond pas très différent.

Un agent est un système où le composant central (le LLM) est non déterministe et non fiable. Toute l'architecture software développée vise précisément à compenser cela et à construire un outil le plus fiable possible.

La logique du cours et des TDs

Énormément de frameworks agentiques qui sortent tous les jours (un des derniers en date : [NVIDIA NemoClaw](#)) mais on peut rapidement se perdre dans tout ça : l'idée est donc de reprendre calmement les agents de zéro, de voir quelles sont les briques de base, et comment on les assemble pour faire quelque chose de fonctionnel. On ne va pas chercher l'efficacité, mais voir comment améliorer et contrôler la performance des agents.



La logique du cours et des TDs (2)

Cela se reflète dans la façon dont les TDs sont structurés :

1. On commence par coder un agent de zéro, avec un LLM local
2. On remplace par un LLM API (limite du local au vu des contraintes GPU du cours) et on découvre le native tool calling via API
3. On ajoute une fonctionnalité de RAG et on regarde comment gérer la mémoire
4. On passe à un framework agentique solide (LangGraph)
5. On ajoute du contrôle (garde-fous, observabilité, human-in-the-loop)



0.1

OpenClaw Teaser

Aller au delà des LLM

Exemple : construire un site web.

Exemple : pour investir dans une entreprise, il y a souvent pleins de dates importantes. Quel système utiliser pour extraire toutes les dates de ces documents ? Les prioriser ? Et les exporter dans un calendrier ?

Avec agent conversationnel :

- nombreuses interactions
- reformulation de prompts
- pas d'action
- pas d'accès à un vrai workspace (python, dossiers)

Au delà, possibilité de

- objectif unique
- autonomie
- exploration
- utilisation d'outils
- itérations automatiques
- livrable final
- Accès à notre système

Exemple avec OpenClaw



> What It Does



Runs on Your Machine

Mac, Windows, or Linux. Anthropic, OpenAI, or local models. Private by default—your data stays yours.



Any Chat App

Talk to it on WhatsApp, Telegram, Discord, Slack, Signal, or iMessage. Works in DMs and group chats.



Persistent Memory

Remembers you and becomes uniquely yours. Your preferences, your context, your AI.



Browser Control

It can browse the web, fill forms, and extract data from any site.



Full System Access

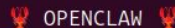
Read and write files, run shell commands, execute scripts. Full access or sandboxed—your choice.



Skills & Plugins

Extend with community skills or build your own. It can even write its own.

OPENCLAW



OpenClaw setup

◇ Security disclaimer

OpenClaw is a hobby project and still in beta. Expect sharp edges. By default, OpenClaw is a personal agent: one trusted operator boundary.

This bot can read files and run actions if tools are enabled. A bad prompt can trick it into doing unsafe things.

OpenClaw is not a hostile multi-tenant boundary by default. If multiple users can message one tool-enabled agent, they share that delegated tool authority.

If you're not comfortable with security hardening and access control, don't run OpenClaw. Ask someone experienced to help before enabling tools or exposing it to the internet.

Recommended baseline

- Pairing/allowlists + mention gating.
- Multi-user/shared inbox: split trust boundaries (separate gateway/credentials, ideally separate OS users/hosts).
- Sandbox + least-privilege tools.

Exemple avec OpenClaw : choix du LLM provider

Model/auth provider

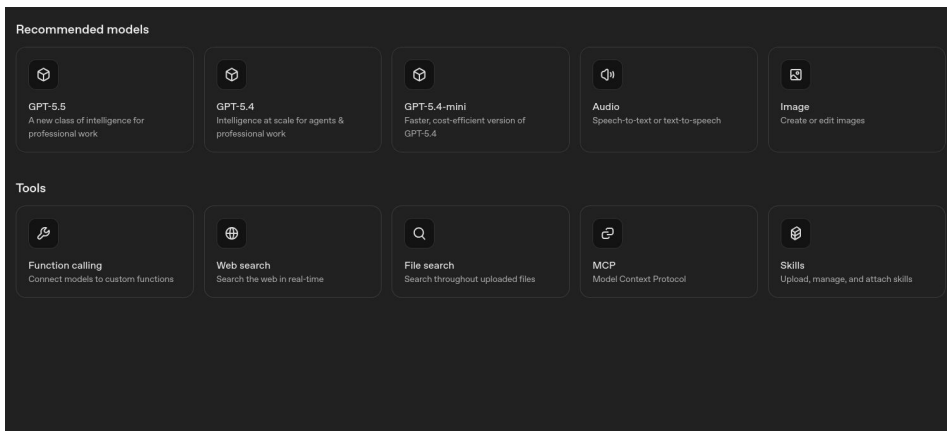
Search:

...

- ☐ BytePlus
- ☐ Cerebras
- ☐ Chutes
- ☐ Cloudflare AI Gateway
- ☐ Codex
- ☐ Copilot
- ☐ Custom Provider
- ☐ DeepInfra
- ☐ DeepSeek
- ☐ Fireworks
- ☐ Google
- ☐ Google Vertex
- ☐ Groq
- ☐ Hugging Face
- ☐ Kilo Gateway
- ☐ LiteLLM
- ☐ LM Studio
- ☐ Microsoft Foundry
- ☐ MiniMax
- ☐ Mistral AI
- ☐ Moonshot AI (Kimi K2.6)
- ☐ NVIDIA
- ☐ Ollama
- ☒ OpenAI (Direct API key)
- ☐ OpenAI Codex
- ...

On choisit OpenAI parce que c'est le plus simple

- stable
- bon tool calling
- JSON/schema robustes
- setup simple



Exemple avec OpenClaw : choix du modèle de LLM

Default model

Search:

- ☐ Keep current (openai/gpt-5.5)
- ☐ Enter model manually
- ☐ openai/gpt-4
- ☐ openai/gpt-4-turbo
- ☐ openai/gpt-4.1
- ☐ openai/gpt-4.1-mini
- ☐ openai/gpt-4.1-nano
- ☒ openai/gpt-4o (GPT-4o · ctx 125k · API key route)
- ☐ openai/gpt-4o-2024-05-13
- ☐ openai/gpt-4o-2024-08-06
- ☐ openai/gpt-4o-2024-11-20
- ☐ openai/gpt-4o-mini
- ☐ openai/gpt-5
- ☐ openai/gpt-5-chat-latest
- ☐ openai/gpt-5-mini
- ☐ openai/gpt-5-nano
- ☐ openai/gpt-5-pro
- ☐ openai/gpt-5-codex
- ☐ openai/gpt-5.1
- ☐ openai/gpt-5.1-chat-latest
- ☐ openai/gpt-5.1-codex
- ☐ openai/gpt-5.1-codex-max
- ☐ openai/gpt-5.1-codex-mini
- ☐ openai/gpt-5.2
- ☐ openai/gpt-5.2-chat-latest
- ☐ openai/gpt-5.2-codex

Exemple avec OpenClaw : choix du modèle de LLM

```
Lis AGENT.md, puis exécute la tâche sur input_docs/.
```

```
Let's proceed with the task step-by-step for mining deadlines. Here's the plan:
```

1. Analyze all documents in the `input_docs/` directory to extract investor-relevant deadlines.
2. Create `outputs/deadlines.csv` using the required CSV schema.
3. Create `outputs/extraction_report.md` summarizing the deadlines and any uncertainties.
4. Run validation on the CSV and make adjustments if necessary.
5. Convert the CSV into an ICS file to be used as an investor calendar.

```
Let's start by analyzing the documents in input_docs/. I'll begin by reading through those files. It seems that the input_docs directory doesn't exist. Please verify that the directory and the documents are in the correct location. Once they're available, I can proceed with the analysis.
```

```
run error: ⚠Rate limit reached for gpt-4o in organization org-r9Uj9P1t65yAvq0mae6Q0Xrt on tokens per min (TPM): Limit 30000, Used 18277, Requested 13690. Please try again in 3.933s. Visit https://platform.openai.com/account/rate-limits to learn more.
```

```
This response is taking longer than expected. Send another message to continue.
```

```
local ready | idle
```

```
agent main | session main | openai/gpt-4o | tokens 12k/128k (9%)
```

- J'ai dépassé le quota de 30000 tokens par minutes
- Montre les limitations possibles d'utiliser un modèle en API
- On peut modifier le prompt (ex. analyse uniquement un document à la fois) mais ce n'est pas une vraie solution
- Changer le modèle dans OpenClaw
- Avoir un modèle en local

Modèle local avec OpenClaw

```
Pour un modele local :  
```bash  
ollama pull qwen3.5
ollama launch openclaw --model qwen3.5
```  
  
```bash  
openclaw gateway restart
```
```

Et ensuite ?

- OpenClaw est quasiment un outils no-code qui permet de rapidement prototyper des agents.
- Idéal pour MVP, validation de cas d'usages et pédagogique pour se familiariser avec le monde des agents.
- Mais abstraction élevée et donc debuggage parfois un peu opaque + pas de contrôle fin, ce qui est nécessaire pour industrialisation de l'agent.
- Fait un peu peur aussi question sécurité : <https://docs.openclaw.ai/gateway/security>
- Reste du cours = au delà du framework comprendre les tenants et aboutissant derrière un agent.



1

QU'EST-CE QU'UN AGENT ?

Agents IA : buzzword ou nouveau paradigme ?

Aujourd'hui on ne parle (presque) plus des LLMs, mais des **agents** : frameworks, MCP, OpenClaw, etc. L'écosystème des agents explose !

Objectifs du cours :

- Comprendre les concepts qui se cachent derrière
- Connaître les éléments essentiels pour un agent
- Coder nos premiers agents from scratch et découvrir les frameworks clés en main
- Faire un assistant de fichiers en local !



ET
NOW
ETNOW.IN

BUSINESS NEWS

**'AI AGENTS TO DRIVE
NEXT TECH REVOLUTION'**



TW
TheCodeWork

THE AI NEWS | THE CODEWORK

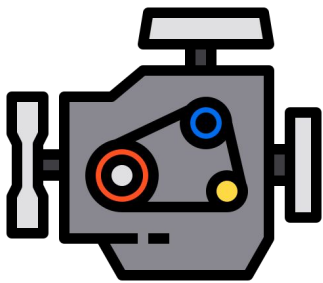
“
The **IT department** of every
company is going to be the
**HR department of AI
agents** in the future.”

NVIDIA CEO Jensen Huang at CES 2025

Pourquoi parle-t-on autant d'agents ?

Avant

LLM → génération de texte
(le prompting + software
tools comme RAG etc.)



Après

Agents → raisonnement +
actions : promesse de
l'automatisation de la GenAI



Pourquoi parle-t-on autant d'agents ?

Un agent, c'est simplement un **LLM** auquel on a rajouté des briques software afin de lui permettre d'**agir sur son environnement**. Par exemple, rechercher des documents, appeler une API, écrire du code ou bien interagir avec un système.

**Agent = LLM + Tools + Memory + Control Loop
+ Environnement + Contraintes**

Frameworks agentiques vs from scratch

On peut coder un agent soi-même en Python : on plug in un LLM, on écrit le code d'orchestration, on gère tout soi-même (les instructions, les tools, la mémoire, les garde-fous, etc.)

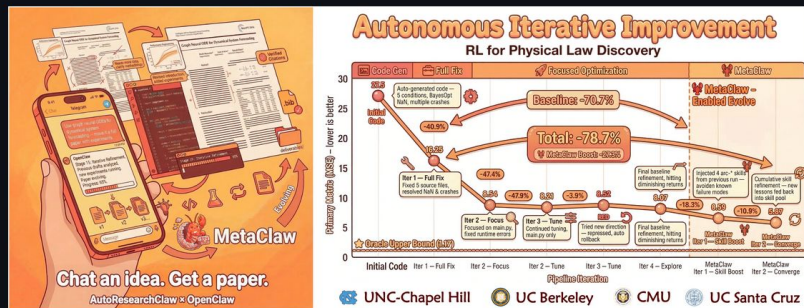
Comme on va le voir, c'est faisable mais pas évident, car il y a beaucoup de travail en pratique pour que les choses fonctionnent bien.

L'autre option c'est d'utiliser des frameworks agentiques pré-existants. Claude Code, OpenClaw sont parmi les plus connus aujourd'hui. Ils packagent des outils, de la gestion de mémoire, des prompts, etc.



Chat an Idea. Get a Paper. Fully Autonomous & Self-Evolving.

Just chat with [OpenClaw](#): "Research X" → done.



License [MIT](#) Python [3.11+](#) Tests [1634 passed](#) GitHub [AutoResearchClaw](#) OpenClaw [Compatible](#)

[Discord](#) [Join Community](#)

Language: [English](#) | [简体中文](#) | [繁體中文](#) | [日本語](#) | [한국어](#)

Everything Claude Code

stars [93k](#) forks [12k](#) contributors [101](#) [ecc-universal](#) weekly downloads [702/week](#) [ecc-agentshield](#) weekly downloads [580/week](#)

[GitHub App](#) [150 installs](#) license [MIT](#) [Shell](#) [TypeScript](#) [Python](#) [Go](#) [Java](#) [Perl](#) [Markdown](#)

50K+ stars | 6K+ forks | 30 contributors | 5 languages supported | Anthropic Hackathon Winner

[Language](#) / 语言 / 語言

[English](#) | [简体中文](#) | [繁體中文](#) | [日本語](#) | [한국어](#)

The performance optimization system for AI agent harnesses. From an Anthropic hackathon winner.

Not just configs. A complete system: skills, instincts, memory optimization, continuous learning, security scanning, and research-first development. Production-ready agents, hooks, commands, rules, and MCP configurations evolved over 10+ months of intensive daily use building real products.

Works across **Claude Code**, **Codex**, **Cowork**, and other AI agent harnesses.

[README](#) [Code of conduct](#) [MIT license](#)

Superpowers

Superpowers is a complete software development workflow for your coding agents, built on top of a set of composable "skills" and some initial instructions that make sure your agent uses them.

How it works

Get Physics Done (GPD)

Built by physicists, for physicists

tests [passing](#) License [Apache 2.0](#) python [3.11+](#) npm [v1.1.0](#)

Get Physics Done is an open-source AI copilot for physics research from [Physical Superintelligence PBC \(PSI\)](#), released as a community contribution. GPD helps turn a research question into a structured workflow: scope the problem, plan the work, derive results, verify them, and package the output.

Les couches du stack agentique (LLM + harness)

LLM : le modèle à la base. Prompt → réponse en texte.

Agent framework : SDK permettant de développer des agents.

Exemples : Agents SDK (OpenAI), LangGraph, CrewAI

Agent platform/runtime : logiciel d'agent, tout packagé, il suffit juste de rajouter une clé d'API et éventuellement un MCP sur sa base de connaissances. Exemples : Claude Code, OpenClaw, Cursor, Codex, OpenHands, Devin, Windsurf

MCP/tool ecosystem : les scripts, plugins, bases de données, API auxquelles les agents peuvent accéder

Les couches du stack agentique



LLM

Le modèle à la base

Prompt → réponse en texte

- Comprend le langage naturel
- Génère du texte
- Ne connaît pas vos outils ou données
- Exemples : GPT-4o, Claude 3, Llama 3



Exemples : OpenAI, Anthropic, Meta



Agent framework

SDK pour développer des agents

Orchestre les appels au LLM, la mémoire, les tools, les workflows

- Abstractions pour construire des agents
- Gestion de l'état, des étapes, des boucles
- Intégration facile de tools/plugins
- Exemples : Agents SDK (OpenAI), LangGraph, CrewAI



Exemples : OpenAI Agents SDK, LangGraph, CrewAI



Agent platform / runtime

Logiciel d'agent tout packagé

Il suffit d'ajouter une clé d'API et éventuellement un MCP sur sa base de connaissances. L'agent est prêt à l'emploi.

- Interface utilisateur
- Mémoire, RAG, tools intégrés
- Connecteurs, permissions, sécurité
- Exemples : Claude Code, OpenClaw, Cursor, Codex, OpenHands, Devin, Windsurf



Exemples : Claude Code, OpenClaw, Cursor, Codex, OpenHands, Devin, Windsurf



MCP / tool ecosystem

Rajouter des capacités à l'agent

Connecteurs vers des outils, APIs, bases de données, services externes.

- Donne des capacités concrètes à l'agent
- Accès au monde réel (web, fichiers, APIs, etc.)
- Standards : MCP (Model Context Protocol), APIs, plugins, custom tools



Web



Fichiers



Bases de données



Calendrier



Email

Exemples : Web search, Calendar, Gmail, Notion, Slack, GitHub, Bases de données, etc.

Les platforms agentiques

On peut distinguer entre **open-source** (OpenClaw, OpenHands, etc.) et **closed-source** (Claude Code, Cursor, Codex, etc).

On peut distinguer aussi en fonction de l'objectif :

- Coding agents = Cursor, OpenClaw, Claude Code etc.
- General productivity agents = ChatGPT Agents, Gemini Agents
- Entreprise = LangGraph platforms, CrewAI enterprise etc.

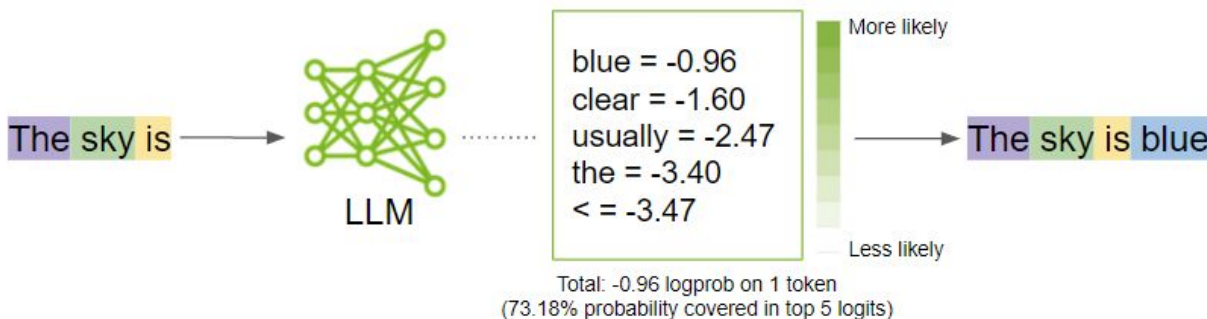


1.1

De LLMs à Agents

Une piqûre de rappel sur les LLMs

- Un LLM est un gros modèle d'IA capable de capturer la statistique complexe des tokens de texte, entraîné sur une quantité astronomique de texte.
- LLMs génératifs entraînés en *next-token prediction*
- LLM = au fond ça reste du prompt : interaction constante nécessaire avec l'utilisateur (chat) mais pas d'automatisation



Une piqûre de rappel sur les LLMs (2)

Texte → tokenizer → LLM → probabilité du prochain token

- Apprentissage auto-supervisé
- Next-token prediction
- Pas de mémoire persistante
- Pas d'accès au monde extérieur autrement que via l'utilisateur

Un LLM seul ne sait pas agir !

Là où c'est trompeur...

En fait, quand on interagit avec un LLM via chat aujourd'hui, on interagit avec **un agent (bridé)**: ChatGPT et cie ne font évidemment pas que de la génération de texte sur la base de leur connaissance interne !

Ils ont accès à des tools (par ex. faire une requête Google) et à des documents/sites web pré-indexés pour faire du RAG.

ChatGPT aujourd'hui = un agent conversationnel capable d'interagir avec Internet. Mais le cadre reste fortement contrôlé par **sandboxing** (isolation technique) et **HITL implicite** (contrôle humain structurel).

Ce n'est pas vrai des LLMs en API qui eux restent des LLMs "traditionnels" !

L'écosystème LLM

Écosystème complexe développé autour des LLMs depuis quelques années :

- Modèles open-source/closed source
- APIs et interfaces de chat
- Briques software (RAG, reranking, etc.)
- Frameworks (LangChain, CrewAI)
- Moteurs d'inférence (Ollama, vLLM, etc.)

→ les agents sont une extension naturelle de cela !

Des LLMs aux agents

Les agents IA sont des systèmes où un modèle de décision (souvent un LLM) interagit avec un environnement externe (fichiers, logiciels, APIs, documents) grâce à des outils, de la mémoire et des contraintes de contrôle.

Un agent = une boucle autonome

perception → raisonnement → décision/action → mise à jour d'état

L'utilisateur définit un objectif.

Pourquoi maintenant ?

Ca ne fait pas si longtemps (2024) que la GenAI a “transitionné” vers les agents. Pourquoi ?

- LLMs suffisamment capables : meilleure compréhension des instructions, planification implicite, robustesse multi-étapes.
- Tool calling fiable : on a mis en place des outils pour éviter le parsing peu robuste des outputs de LLMs.
- Grandes context windows : essentielles pour gérer instructions/catalogue de tools/mémoire efficacement.
- Écosystème LLM riche : RAG, moteurs d'inférence, vector databases etc.
- Coûts en baisse (les agents demandent BEAUCOUP de tokens).



1.2

Conceptuellement

Philosophie des agents

Faire des actions : au sein d'un agent, les LLMs peuvent accomplir des tâches en interprétant la query de l'utilisateur et en utilisant les outils disponibles dans leur environnement.

Accès à des outils : on va spécifier au LLM des outils auxquels il peut faire appel : des fonctions lui permettant d'agir sur son environnement (lire, modifier, exécuter)

Mémoire et base de connaissances : l'agent va être capable de reasoning car on lui organise une mémoire court-terme (comme dans un chat) et long-terme (en laissant l'agent se construire une base de connaissances issues de son interaction avec son environnement).

Comment fonctionne un agent ?

perception → raisonnement → décision/action → mise à jour d'état

Programme qui boucle :

1. l'agent **observe** une situation (via prompt et mémoire) ;
2. il **décide** d'une action (via le modèle de décision (souvent un LLM)) ;
3. il **exécute** cette action (via *tool calling*)
4. Il **met à jour son état interne** (mémoire, historique, résultats) via résultat du tool calling et politique de mise à jour définie extérieurement

Un agent produit implicitement du texte (comme un LLM simple) mais de manière **structurée, formatée, standardisée**, et qu'on peut donc directement reliée à l'exécution d'un outil (pas sorcier !)

Les briques de base de l'agent

1. État : ce que l'agent "sait" et doit conserver entre les étapes de la boucle
2. Outils : les fonctions auxquelles l'agent a accès et qu'il peut donc appeler
3. Politique de décision (*reasoning loop*)
4. Mémoire (le contexte)
5. Boucles et garde-fous

Différences entre chat, workflow et agent

Chat

Query → réponse
générée via next-token
prediction

Workflow

Query → database (ex.
RAG) → réponse, étapes
déterministes

Agent

Query → réflexion →
sélection d'outil →
observation → décision,
en boucle

L'architecture cognitive de l'agent

1. **État** = Ensemble structuré des informations internes disponibles pour l'agent a un instant donné. L'état n'est pas forcément tout envoyé au LLM
2. **Contexte** = sous-ensemble de l'état injecte dans le modèle lors d'un appel donné
3. **Mémoire** = la mémoire organise ce qu'il conserve. Mémoire de travail, mémoire persistante etc.

L'état représente tout ce que l'agent sait actuellement ; la mémoire organise ce qu'il conserve ; le contexte sélectionne ce qu'il utilise immédiatement.

Prompting, RAG, agents : quelle différence ?

Prompt engineering : Question → LLM → réponse

Le LLM répond avec ce qu'il sait déjà sans accès à des données externes

RAG : Question → retrieval → contexte → réponse

Le LLM répond en ayant accès à de l'information externe

Agent : Question → LLM reasoning → choix d'action → tool/retrieval → observation → nouvelle décision

Le système décide tout seul de quoi faire (parmi un catalogue d'actions autorisées), peut itérer, et décide de quand s'arrêter.

Ce que nous allons faire dans ce cours

On y va progressivement en essayant d'introduire les concepts essentiels au fur et à mesure :

1. Comprendre et coder un agent basique en API et en local
2. Utiliser des outils de tool calling natif
3. Comprendre les *structured outputs*
4. Gérer la mémoire et le RAG dans les agents
5. Graph agents
6. Observabilité
7. Agents en production



1.3

Les moteurs d'inférence

Moteurs d'inférence

Évidemment dans tout ça, il faut faire tourner le LLM en arrière-plan. Et on veut éviter tout le code qu'on avait développé dans le cours LLM : charger le LLM, écrire la pipeline, etc. La solution c'est d'utiliser un **moteur d'inférence**.

Un moteur d'inférence est le logiciel qui exécute un modèle pour produire des réponses à partir d'un prompt. Il gère concrètement la tokenisation, le passage dans le modèle, et la génération des tokens. Il optimise aussi l'exécution, par exemple en utilisant la GPU, la quantization, le CPU offloading, ou le batching pour aller plus vite.

Exemples classiques : **Ollama**, **vLLM** ou **llama.cpp** en local, ou des APIs comme OpenAI côté cloud (parce qu'une API au fond c'est la même chose, juste sur le cloud)

Ollama



Un runner qui télécharge, configure et sert des modèles locaux avec une interface uniforme. L'idée = "Docker pour LLMs". On fait pull puis run, et hop on a un serveur local pour appeler le modèle en HTTP.

- Très facile à utiliser : `ollama run <model>`
- Setup minimal, très fiable.
- Très bon pour un agent local "single user".
- Large catalogue de modèles
- Modèles quantisés gérés simplement
- Inclut frameworks type Claude Code ou OpenClaw

Peu optimisé pour la performance
(pas idéal pour prod) + Tool
calling non-natif et donc pas très
robuste

Cf. <https://docs.ollama.com/quickstart>

```
## Utiliser ollama
'''bash
sudo apt-get update
sudo apt-get install zstd
curl -fsSL https://ollama.com/install.sh | sh
nohup ollama serve & # runs in background
'''

Pour installer un modèle avec Ollama, faire :

'''bash
ollama pull qwen3.5:9b
'''

Et pour le supprimer :

'''bash
ollama rm qwen3.5:9b
'''

Pour afficher la liste des modèles installés, on peut faire :

'''bash
ollama list
'''

Et pour tester ce modèle :

'''bash
ollama run qwen3.5:9b
'''
```

Le serveur haute performance → pour la production. C'est un moteur d'inférence surtout orienté GPU, optimisé pour le débit et la concurrence (beaucoup de requêtes). Il fournit un serveur HTTP compatible OpenAI (Chat Completions, Completions, etc.).

- Très performant (optimisé pour maximiser le throughput (batching, KV cache))
- Scalable (adapté multi-utilisateurs)
- Compatible avec API OpenAI/Anthropic

Setup plus complexe,
gestion des modèles en local
plus technique que dans
Ollama

Cf. <https://www.redhat.com/fr/topics/ai/vllm-vs-ollama>

le KV Cache

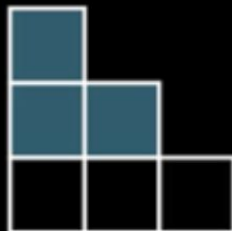
KV Caching Explained



Q[-1]



K^T



Attn

le KV Cache

Idée est très simple :

- Le LLM génère token par token donc beaucoup de calcul répète
- Chaque token demande le calcul de
 - Q = ce que le token cherche
 - **K = un résumé ce que le token propose**
 - **V = le contenu de ce que le token propose**

Donc une bonne approximation est de ne pas recalculer les K , V des anciens tokens. Ce qui aboutit au dilemme entre : réduire les calculs ou stocker en mémoire le cache KV (qui est compressé intelligemment bien sûr)

KV cache = des avancées incroyables

TriAttention: Efficient Long Reasoning with Trigonometric KV Compression

Weian Mao^{1*}

Xi Lin^{3*}

Wei Huang^{2*}

Yuxin Xie¹

Tianfu Fu¹

Bohan Zhuang³

Song Han^{1,2}

Yukang Chen²

Abstract

Extended reasoning in large language models (LLMs) creates severe KV cache memory bottlenecks. Leading KV cache compression methods estimate KV importance using attention scores from recent post-RoPE queries. However, queries rotate with position during RoPE, making representative queries very few, leading to poor top-key selection and unstable reasoning. To avoid this issue, we turn to the pre-RoPE space, where we observe that Q and K vectors are highly concentrated around fixed non-zero centers and remain stable across positions—*Q/K concentration*. We show that this concentration causes queries to preferentially attend to keys at specific distances (e.g., nearest keys), with the centers determining which distances are preferred via a trigonometric

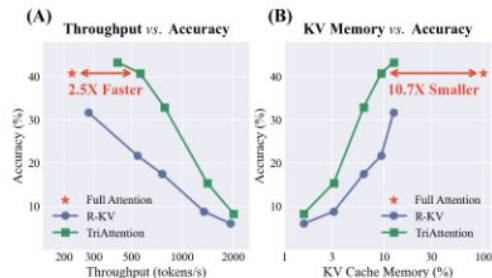


Figure 1. Performance trade-offs on AIME25 (Qwen3-8B). (A) At equivalent accuracy (40.8%), TriAttention achieves $2.5\times$ higher throughput than Full Attention. (B) TriAttention reduces KV cache memory by $10.7\times$ while matching Full Attention accuracy.

[cs.CL] 6 Apr 2026

<https://arxiv.org/pdf/2604.04921>

<https://www.marktechpost.com/2026/04/11/researchers-from-mit-nvidia-and-zhejiang-university-propose-triattention-a-kv-cache-compression-method-that-matches-full-attention-at-2-5x-higher-throughput/>



1.4

Nouveautés en LLMs

Montée des modèles open-weight agentiques

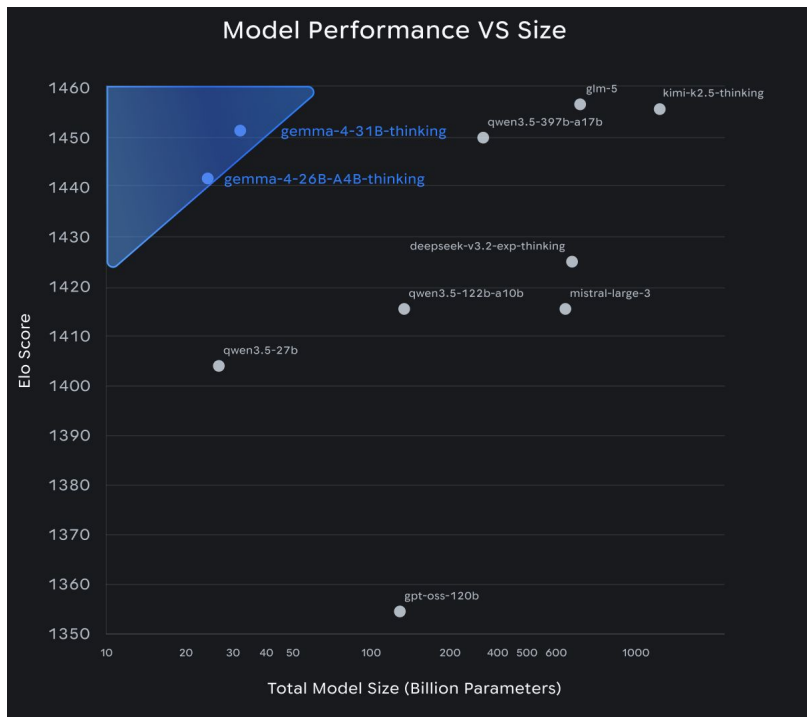
- Les modèles fermes restent très forts, mais l'écart avec open-weight se réduit
- La Chine pousse très fort l'open-weight : deepSeek, Qwen, Kimi, GLM, MiniMax etc.
- Les modèles récents sont de plus en plus agentiques : tool use, coding, long context, multi-step reasoning
- Modèles de plus en plus efficaces en inference (ex. MoE) avec seulement une fraction des paramètres activés à l'inférence

Les modèles open-weight chinois

- **Kimi K2** : MoE avec 1 Trillion parameters, optimisé agent
- **DeepSeek V3** : 671 billion parameters, aussi optimisé agent
- **Qwen3** : 235 billion parameters, MoE, et aussi optimisé agent
- **GLM-5** : same story
- **MiniMax M2** : same story



Et maintenant google !



Gemma 4: Byte for byte, the most capable open models

Apr 02, 2026 · 6 min read



Clement Farabet
VP of Research, Google
DeepMind



Olivier Lacombe
Director Product
Management, Google
Deepmind

Share



Quelle stratégie à envisager ?

La bataille des LLMs se déplace vers les modèles agentiques, efficaces à l'inférence, souvent open-weight, et capables d'être intégrés dans des workflows métier.

- C'est le far-west : certains acteurs voudront vous bloquer de façon propriétaire
- Avec un framework agentic trop no-code, il risque de ne pas être le plus efficace en token pour résoudre nos tâches
- Investir dans la maîtrise un peu plus bas niveau des agents
- Tout repenser (usages, stratégie de boîte) en fonction de cette nouvelle couche techno

Il devient donc essentiel d'investir sur la capacité à créer des agents locaux, efficaces et peu consommateurs en token



2

CONSTRUIRE UN AGENT

L'agent n'est pas le LLM :

c'est un programme qui **orchestre** un ou plusieurs LLMs (ou autres types de modèles)

Le cycle de décision

Rappel : query → LLM → tool calling → observation → mise à jour

1. On donne au LLM : l'objectif (query) / l'état courant / la liste des outils disponibles (nom, description, schema JSON des arguments)
2. Le LLM répond soit "je réponds en texte final" / "j'appelle un outil X avec des arguments Y"
3. Si appel d'outil, l'agent exécute l'outil (validation user éventuelle)
4. On observe le résultat et on l'ajoute dans l'état (ou dans l'historique).
5. On relance le LLM.
6. On s'arrête quand le LLM renvoie une réponse finale ou quand on atteint les limites (max tokens, max steps, etc.)

État d'un agent

L'état est ce qui permet à l'agent de continuer à raisonner d'une étape à l'autre. En pratique, c'est l'information qui vient s'inscrire dans le contexte. Il n'y a rien de mystérieux : in fine à la base d'un agent il y a un LLM qui nécessite un prompt. Tout l'art consiste à bien générer/structurer ce prompt à partir des informations dont dispose l'agent :

- historique de conversation
- résultat(s) des outils
- raisonnements intermédiaires
- mémoire de travail
- informations tirées des documents

TD : app_TD1_base.py

Commençons par illustrer concrètement coder ces boucles agentique

```
# TD1 : coder un Agent de zero avec Ollama (en local) et OpenAI (API).
```

```
Ici, on veut simplement créer un agent qui a accès à une structure de fichiers afin d'illustrer :
```

- comment faire une boucle avec plusieurs appels LLM ;
- comment tester avec un LLM local ou via une API dans la boucle agentique ;
- comment gérer naïvement l'appel à des fonctions Python dans la boucle agentique.

```
En particulier, dans le fichier `src/tools.py`, on définit un ensemble de fonctions permettant d'effectuer des actions sur des structures de fichiers.
```

```
On va donc illustrer la boucle agentique minimale :
```

- construire les messages
- appeler le LLM
- parser la sortie JSON pour appeler une fonction quand nécessaire
- si tool_call: exécuter l'outil (tools.py), injecter le résultat, et boucler
- si final: renvoyer la réponse à l'utilisateur



2.1

Appeler des outils

Les structured JSON (1)

On ne peut pas laisser les LLMs libres de générer n'importe quel type de texte. Le texte "libre" n'est pas facilement parsable... Dans le monde des agents on va donc travailler avec des "structured JSON outputs" (avec schéma strict).

L'idée est d'avoir un format uniforme, clair et facilement parsable pour pouvoir passer d'une étape de la boucle agentique à une autre.

Attention, ca ne veut pas dire que dans les boucles agentiques, tout est du JSON ! Il y a un mix entre texte libre (input utilisateur/output final), JSON pur et formats hybrides.

Les structured JSON (2)

Les structured JSON interviennent à **deux endroits critiques** de la chaîne agentique :

- **Décision du LLM** (tool calling ou réponse finale)

```
{  
  "tool": "read_file",  
  "arguments": {"path": "data/invoice.txt"}  
}
```

- **Observation du résultat des outils**

```
{  
  "ok": true,  
  "content": "...",  
  "metadata": {...}  
}
```

Les structured JSON (3)

- Pas encore de format standard, universel, mais on converge
- Structured JSON : pas simplement du JSON, mais du JSON pensé pour être validé par un schéma (pour pouvoir faire des validations automatiques des JSON → déterministe)
- Le reste est laissé libre (notamment tout le LLM reasoning)

Comment gérer les erreurs de JSON ?

C'est toute la question pour faire des agents robustes !

On ne peut pas supposer que les LLMs vont toujours générer des JSON parfaits : soit JSON cassés, tools incorrect ou inexistant, arguments incorrects, etc.

Cela peut arriver parce que le LLM n'est pas assez bon (cf. TD Ollama), qu'il y a de l'ambiguïté dans les prompts, qu'il manque du contexte, que les outils sont mal définis, que la tâche est trop complexe, etc.

→ partie suivante : observabilité, guardrails, LangGraph

Qu'est-ce qu'un outil ?

Un outil est une **fonction externe** que le LLM peut appeler :

- API
- lecture/retrieval dans base de données
- CLI
- code Python
- ...

Par exemple, dans le TD1 :

- lister les fichiers dans un répertoire
- lire un fichier texte
- chercher une string dans un fichier

Comment définir un outil ?

Typiquement, on va définir un outil en deux parties :

- la fonction elle-même (`def read_file(...)` en Python)
- un dictionnaire (JSON) représentant la fonction (et qui sera repris dans le tool calling) :

```
{  
    "name": "read_file",  
    "arguments": { "path": "string" }  
}
```


Design des tools

C'est un des facteurs essentiels de la qualité de l'agent. Il faut viser :

- une granularité optimale (et expliciter ça au LLM : quel tool appeler dans quelles conditions, pour faire quoi, etc.)
- une nomenclature explicite
- pas d'overlap des tools pour éviter les confusions du LLM
- schéma strict (inputs, outputs)

Exemple bof : donner `search_in_files` et `retrieve_from_documents` (cf. TD) sans expliciter au LLM la différence entre les deux.

Tool execution

Le résultat du tool peut être n'importe quel objet Python (après tout, le tool est une fonction quelconque), qui va être typiquement converti en dictionnaire.

```
{  
    "ok": True,  
    "data": ...,  
    "error": None  
}
```

Le champ "ok" sert à garder une trace de ce que tout se passe bien dans la boucle.

L'idée c'est de rester dans des formats standardisés pour être robuste et interopérable (ce sera le coeur du tool calling natif)

Tool arbitration

Quand plusieurs tools sont plausibles, le LLM peut choisir le mauvais, hésiter, ou bien se diriger vers une stratégie suboptimale. Exemple simple :

Q: What does Globex buy from us?

A (mauvais) : `search_in_files("What does Globex buy from us?")`

A (bonne) : `search_in_files("Globex") puis read_file(...)`

Alors comment faire ? En pratique :

- décrire les tools avec beaucoup de détails
- éviter l'overlap des tools en choisissant des tools très spécialisés
- réduire le nombre de tools disponibles

→ on verra plus de détails dans la partie observabilité/contrôle/LangGraph

Gérer les JSONs, les tool, le parsing...

On comprend que si on veut faire tout ça à la main, c'est la galère :

- il faut des règles de parsing claires et robustes : beaucoup de travail pour réinventer la roue
- il faut contraindre le LLM à respecter ces règles, donc rédiger des instructions dans le prompt : là encore, pourquoi galérer alors que ce sont des choses a priori très standard ?

La solution : le **native tool calling**



2.1

Appeler des outils

Native tool calling

On passe de LLM → texte → parsing JSON → exécution (fragile) à
LLM → tool call structuré → exécution directe (robuste)

Concrètement, on va fournir une liste
de tools :

```
{  
  "name": "read_file",  
  "description": "...",  
  "parameters": { JSON schema }  
}
```

Côté API, tout ca est injecté dans le
contexte du modèle sous forme structurée.
Le LLM choisit une action et va renvoyer :

```
{  
  "tool_calls": [  
    {  
      "name": "read_file",  
      "arguments": {"path":  
        "data/file.txt"}  
    }  
  ]  
}
```

Exercices: créer vos tools

Voir le jupyter notebook : `exercice_1_tool_calling_LLMs.ipynb`

5. Exercice : écrire un outil pour répondre à une question difficile sans calcul fiable

On souhaite répondre au prompt suivant :

J'ai acheté 3 articles :

- un clavier à 79,90 €
- une souris à 39,50 €
- un écran à 249,99 €

J'ai un code promo de 12 %, puis je dois ajouter une TVA de 20 %. Les frais de livraison sont de 8,90 €, mais ils sont offerts si le total TTC avant livraison dépasse 350 €.

Quel est le montant final à payer ?

Native tool calling

Au fond rien de plus compliqué que la combinaison de :

- méthodes d'injection automatique des JSON dans les prompts
- LLMs fine-tunés sur des instructions contenant du JSON et forcés à produire du JSON structure en sortie
- processus de validation de la sortie côté API

Aujourd'hui standard dans les APIs agentiques (OpenAI, Anthropic etc.)

Peu robuste en local (il faut un LLM qui gère ça bien = qui sait se discipliner à produire des JSON structures en sortie... pas évident !)

Implémenté dans les frameworks open-source type Agent SDK, LangGraph

Quels LLMs pour native tool calling ?

Les LLMs en API sont robustes pour le native tool calling parce qu'ils ont été fine-tunes spécifiquement pour ça ! En plus, OpenAI et cie. ont fine-tuné leurs LLMs sur des tools très variés pour garantir leur généralisabilité à la plupart des tools que les utilisateurs vont leur passer.

Avec des LLMs en local, on ne peut pas couper à un peu de parsing à la main, ou alors faire du fine-tuning 😡

C'est pour ça que certains poussent pour les CLI comme base du tool calling/MCP dans les agents

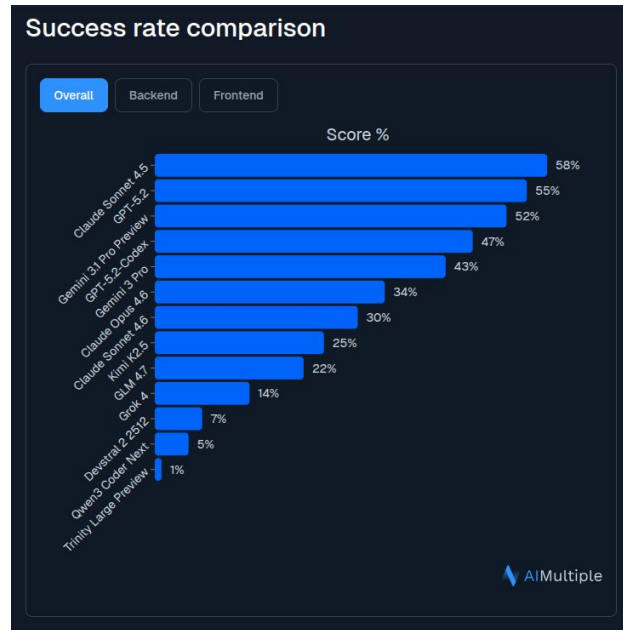
<https://huggingface.co/learn/agents-course/bonus-unit1/what-is-function-calling>

https://docs.mistral.ai/capabilities/function_calling

En résumé

Le comportement de l'agent dépend fortement :

- du LLM (instruction tuning, reasoning)
- du prompt
- du tool calling behavior



<https://aimultiple.com/agentic-llm>

TD : app_TD2_tool_calling.py

On revisite app_TD1_base.py en remplaçant la gestion un peu bricolée des fonctions renvoyant des jsons par :

- On fournit la liste des tools directement à API openAI
- On change les procédures de vérification de l'output du LLM dans la boucle



2.2

Les skills

Les skills

Une skill, c'est une notice opérationnelle qui explique à l'agent comment réaliser une tâche spécialisée, éventuellement en utilisant certains fichiers ou ressources.

Et on peut simplifier cela par exemple comme une notice (un SKILL.md) associé à un dossier de fonctions qui sont décrits dans le .md

```
my-skill/  
  SKILL.md  
  scripts/  
    ├── build_ics.py  
    └── validate_deadlines.py  
  references/  
  assets/
```

Un tool est une capacité atomique. Une skill est une compétence composée.

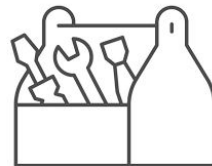
Les skills

Une skill sert à rendre réutilisable et propre un comportement. Quand l'utilisateur demande ce type de tâche, voici la procédure, les fichiers à produire, les scripts à utiliser, les contraintes.

```
SKILL.md      → instructions métier + règles + workflow  
bin/script.py → scripts/outils locaux
```



Une skill c'est une forme de mémoire procédurale spécialisée



Les skills

Dans beaucoup de cas, une skill peut être vue comme une notice d'orchestration de tools. Mais elle peut aussi être utile même sans tool, simplement comme procédure, checklist, style guide ou méthode de travail, en quelque sort un super prompt...

Une skill doit être liée à un workflow clair répétable :

"Transformer une maquette Figma en composant React"

mais évidemment pas

"Être plus intelligent"

Awesome Claude Skills



awesome

Claude Skills

>> CLICK HERE TO UPGRADE YOUR WORKFLOW

WITH  Composio



awesome

PRs

welcome

License

Apache 2.0

 FOLLOW ON X

 FOLLOW ON LINKEDIN

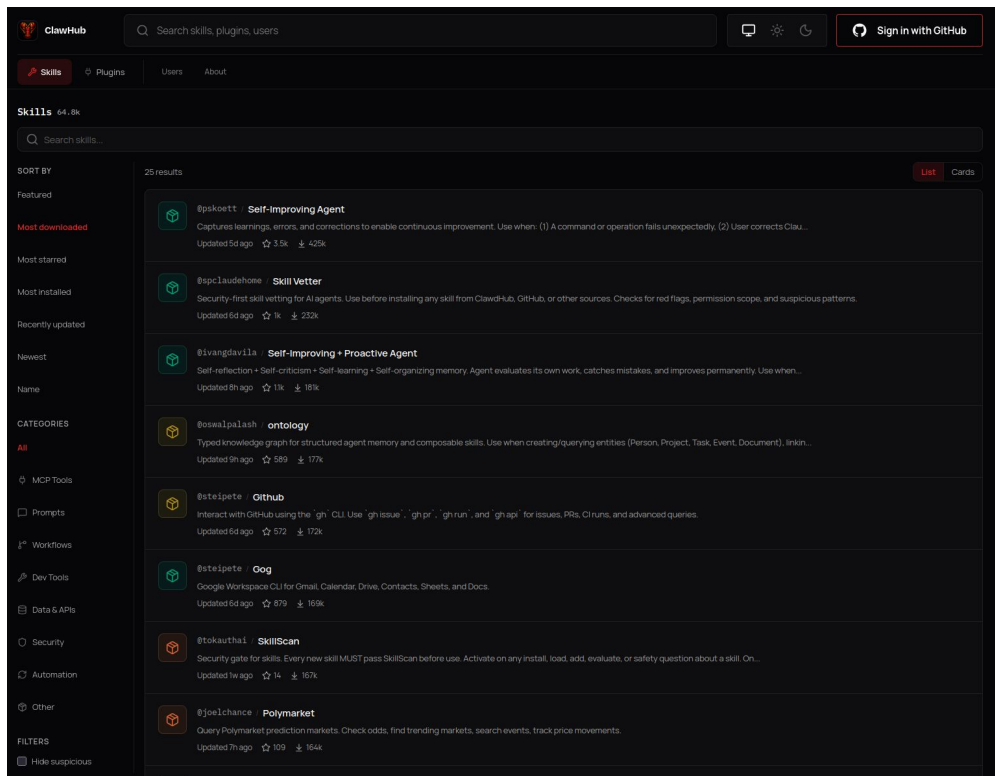
 JOIN OUR DISCORD

A comprehensive and curated list of 1000+ production ready and practical Claude Skills and Plugins for enhancing productivity across usecases on not just Claude.ai, Claude Code, but also across coding agents like Codex, Cursor, Gemini CLI, Antigravity and more.

Want skills that do more than generate text? Claude can send emails, create issues, post to Slack, and take actions across 1000+ apps. [See how →](#)

ClawHub

ClawHub = registre public de skills/plugins OpenClaw.



Caveman skill

[README](#) [Contributing](#) [MIT license](#)



caveman

why use many token when few do trick

stars **57k** last commit **today** license **MIT**

[Before/After](#) • [Install](#) • [What You Get](#) • [Benchmarks](#) • [Full install guide](#)

A [Claude Code](#) skill/plugin (also Codex, Gemini, Cursor, Windsurf, Cline, Copilot, 30+ more) that makes agent talk like caveman — cuts ~75% of output tokens, keeps full technical accuracy. Brain still big. Mouth small.

Before / After

 **Normal Claude (69 tokens)**

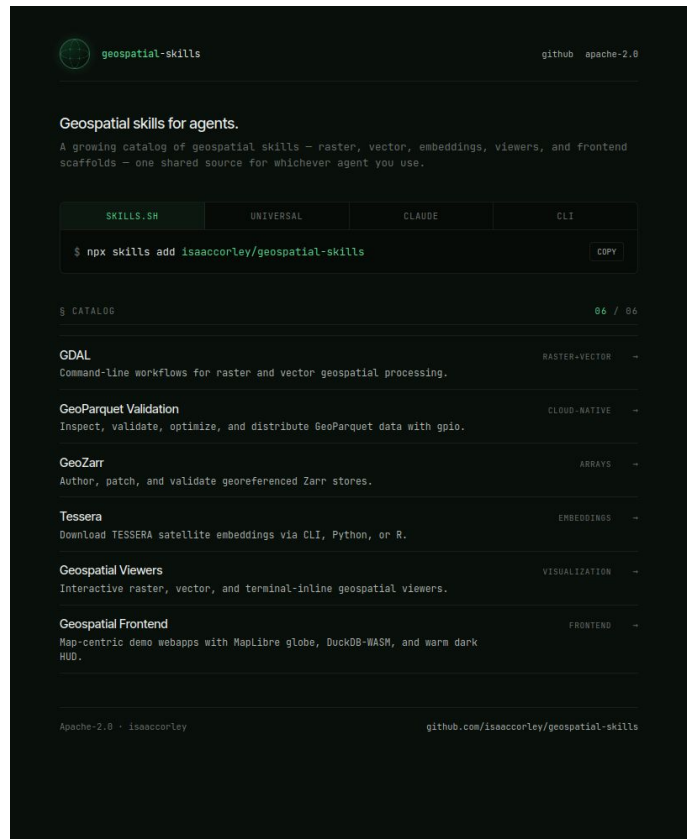
"The reason your React component is re-rendering is likely because you're creating a new object reference on each render cycle. When you pass an inline object as a prop, React's shallow comparison sees it as a different object every time, which triggers a re-render. I'd recommend using useMemo to memoize the object."

 **Caveman Claude (19 tokens)**

"New object ref each render. Inline object prop = new ref = re-render. Wrap in `useMemo`."



Des skills d'agents géospatiaux (open-source)



- Pour apprendre, c'est très utile. Pour installer en entreprise, il faut auditer.
- Risque importants : prompt injection, exécution de code, data leak, dépendances compromises
- Attention, même sur platform, risques de skills malicieuses
- Pas de baguette magique pour auditer les skills = travail
- Faire ses propres skills



2.4

Le LLM dans l'agent

Un agent peut-il utiliser n'importe quel LLM ?

En pratique, **non**. Un modèle adapté aux agents a tendance concrètement à :

1. Respecter strictement des formats structurés
→ JSON propre, pas de texte parasite
2. Suivre des instructions longues et contraignantes
→ "ne réponds qu'en JSON", "appelle un outil si nécessaire"
3. Reasonner en plusieurs étapes
→ planifier, appeler un outil, intégrer le résultat
4. Être stable en boucle
→ ne pas dériver, ne pas halluciner d'outils inexistantes
5. Savoir décider quand appeler un outil
→ ni trop, ni pas assez

Ce sont des **propriétés comportementales** que tous les LLMs n'ont pas !

Pourquoi certains LLMs sont plus adaptés ?

Essentiellement du fait du fine-tuning !

- **Instruction tuning** : le LLM est fine-tuné spécifiquement sur du suivi d'instructions, des formats stricts, des tâches multi-étapes = LLM discipliné
- Fine-tuning avec tool calling : avec des exemples de function calling, JSON schéma, appels d'outils, etc. Le LLM a appris à produire le format JSON de tool calling proprement.
- Fine-tuning avec dialogues multi-tours : les LLMs entraînés avec des dialogues complexes multi-tours gèrent mieux le régime de boucle qu'on demande à l'agent de réaliser.
- Capacité de raisonnement : les gros LLMs ou les LLMs mieux fine-tunés planifient mieux, utilisent mieux le contexte, et/ou font moins d'erreurs de logique.

Un LLM peut être parfait pour du chat (créatif, fluide etc.) mais nullissime comme base d'agent !

Construire le prompt de base

Super important. Il faut mettre le plus d'infos métier possible et tester, tester, tester. C'est comme du prompt engineering pour les LLMs, sauf que là il faut que ca marche bien avec votre setup local :

- faire comprendre à l'agent quelle est la structure de vos fichiers
- lui faire comprendre les tools à disposition et ce qu'ils font
- et toute autre information complémentaire utile !



3

GÉRER LA MÉMOIRE ET DOCUMENT INDEXING

Les limites à ce stade

On a vu comment implémenter un agent simple : LLM + tools + boucle.

Pas mal, mais :

- pas de mémoire autre que l'historique des conversations/tool calls/résultats
- contexte limité
- pas de RAG !

Conclusion : la gestion des fichiers est **inefficace** et l'agent ne peut pas **accumuler de connaissances et la consulter efficacement**.

Les deux points essentiels à rajouter sont donc : **mémoire + retrieval**



3.1

Mémoire et agents

Les différents types de mémoire dans un agent

Mémoire long-terme

Notes/informations
persistantes

Préférences utilisateur
(high-level instructions)

Résultats essentiels stockés

→ mémoire **persistante** entre sessions (= différents inputs = différentes boucles)

Mémoire de travail

Historique de conversation

Résultats récents (tool calls,
tool outputs)

Etat courant (dernier tool
call/prompt etc.)

→ mémoire **temporaire**, limitée
au contexte

Politiques de mise à jour de la mémoire

On ne peut pas tout stocker (tokens = \$\$ et puis fenêtre de contexte limitée quand même !). Il faut donc décider : quoi garder ? quand/où le garder ? quoi oublier ?

Mémoire de travail (épisodique) : "simple"

- garder les N dernières étapes (messages, tool calls, résultats)
- résumer les interactions périodiquement pour compresser et inclure les résumés
- filtrage éventuel pour ne garder que les informations importantes (enlever certains détails de reasoning par exemple).

Mémoire persistante

Pour la mémoire persistante, ça devient plus intéressant. Il faut une politique d'écriture de règles utiles et réutilisables, et on peut vite s'y perdre. Il n'y a pas de *best practice* ici, tout évolue vite et chaque framework va avoir sa propre recette.

La plupart des frameworks fonctionnent avec une stratégie double : **implicit et explicit memory updating**.

Explicit : l'utilisateur fournit un fichier écrit à la main où il stocke des instructions/informations high-level (exemple : **Claude.md**). Parfois possible de stocker explicitement via l'agent (# shortcode in Claude Code)

Implicit : dans ses instructions, l'agent a accès à des règles de sauvegarde et d'oubli : **LLM-as-a-judge**.

Exemples d'instructions persistantes

Typiquement dans memory.md / CLAUDE.md pour un assistant local sur fichiers, on veut indiquer :

- structure des dossiers
- conventions de nommage
- où chercher quoi

Le LLM est puissant, mais sans contexte explicite, il reste aveugle au système de fichiers... C'est toujours utile d'explicitier la nature du monde dans lequel on veut que l'agent vive !

<https://github.com/ArthurClune/claude-md-examples>

<https://agents.md/>

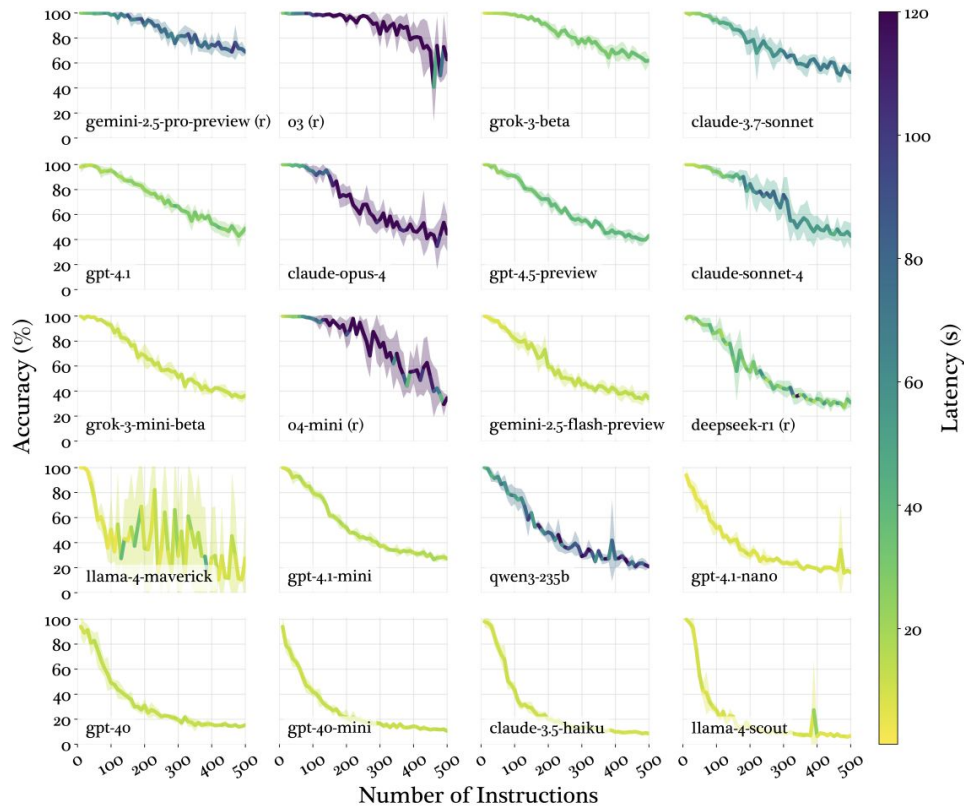
→ équilibre entre complexité/niveau de détail et puissance du LLM

La difficile gestion de la mémoire

Pas possible de toujours ajouter des choses au contexte : le gros risque = se retrouver avec du "clutter" dans les instructions, et ça finit en catastrophe.

Exemple : Claude Code a déjà 50 instructions de base dans son prompt (avant même de prendre en compte l'input utilisateur !) : où sont stockés les fichiers mémoire ou les instructions pour les CLI shortcuts, dans quel ordre lire les fichiers de base etc. Exemple :

```
IMPORTANT: this context may or may not be relevant to your tasks. You should not respond to this context unless it is highly relevant to your task.
```



Implicit memory updating

Eviter trop de contexte : mémoire persistante et épisodique stockées dans .md qui sont indexés (ie chunked + embedded) pour être utilisés via RAG. Eventuellement master memory file qui lui-même indexé les autres fichiers mémoires et qui est chargé par défaut dans le contexte

Equilibre difficile à trouver entre stocker des informations vs. trop de mémoire dégrade le système (hallucinations, confusion, bruit)

Dès qu'on sort des frameworks pré-codés type Claude Code, alors il faut faire ce travail soi-même. Agents SDK, smolagents et cie. gèrent la mémoire épisodique (~ le prompt), pas la mémoire persistante.

Implicit memory updating en pratique

3 briques essentielles pour gérer les fichiers mémoire :

- Politique de sauvegarde explicite dans le prompt
- Outil(s) de mémoire : `remember_note(text)`, `search_memory(query)` ou `delete_memory(id)`
- Contrôle applicatif : override le LLM pour éviter les bêtises. Le LLM propose, le code décide : réduire la taille d'une note trop longue, supprimer les doublons, limite sur la quantité d'informations, structurer les notes en JSON, etc.

Quid de la longueur du prompt ?

Si on ne fait pas attention, on peut également rapidement se retrouver avec des prompts trop longs qui dépassent la taille de la fenêtre de contexte (et qui finissent par nous coûter cher : min. \$1 / million d'input tokens)

Il est donc important de gérer cet aspect, en séparant le contexte actif (= ce qu'on injecte dans le prompt), et une mémoire dormante, hors prompt, relue si besoin.

Comment font les agents opérationnels ?

Ils combinent en général plusieurs stratégies :

- fenêtre glissante : seuls les derniers échanges sont conservés
- compression des conversations : régulièrement, on utilise un LLM pour résumer l'historique des échanges
- les faits utiles sont stockés dans la mémoire persistante
- on utilise un fichier memoire type memory.md pour garder trace de tout ce qui s'est passé, et faire du retrieval a la demande (soit via RAG, soit via recherche plus classique dans mémoire structurée via keywords)



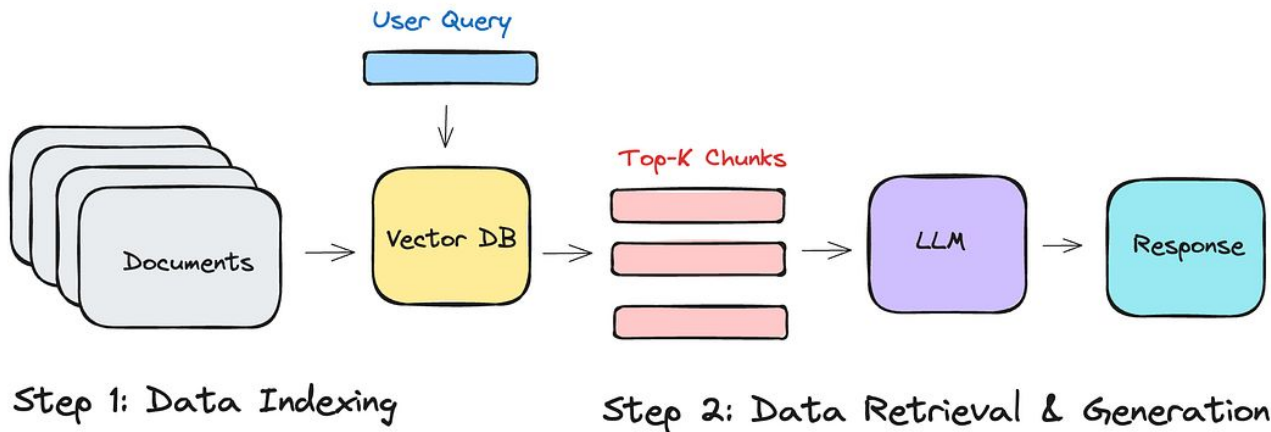
3.2

Accéder à la mémoire

Rappel du RAG

Bases du RAG : chunking + embedding + stockage dans db + retrieval efficace sur la base de l'embedding d'une query (+ éventuel reranking).

→ **accès dynamique à une base de documents**



RAG dans agent

RAG $\neq$ mémoire : c'est un accès aux documents alors que la mémoire correspond à un stockage interne (dans le prompt, les instructions, etc.)

La différence par rapport au RAG "statique" c'est que l'agent va décider tout seul quand faire du retrieval (cela correspond à un tool disponible)

Les "hyperparamètres" classiques du RAG sont toujours valides : stratégie de chunking, chunk size, overlap, embedding model, top-k, application de reranking, etc. → seule l'expérience et éventuellement la nature des documents (chunking "natif") peuvent vraiment permettre de décider

RAG dans agent

Pas plus compliqué que de définir un tool "retrieve_from_document" et d'utiliser les fonctionnalités de FAISS dans l'exécution du tool :

```
"type": "function",
"function": {
    "name": "retrieve_documents",
    "description": "Retrieve top relevant chunks from
the indexed document store using semantic search." ,
    "parameters": {
        "type": "object",
        "properties": {
            "query": {"type": "string"},
            "top_k": {"type": "integer",
"minimum": 1, "maximum": 10},
        },
        "required": ["query"],
        "additionalProperties": False,
    },
}
```

```
if tool_name == "retrieve_documents":
    result =
self.rag_store.retrieve(

query=arguments["query"],

top_k=arguments.get("top_k", 4),
    rerank=False,
)
    if result.get("ok"):

self.working_memory.last_retrieved_chun
ks = result.get("results", [])

    return result
```

TD : app_TD3_with_rag.py

Pour tester en pratique la façon dont la mémoire influe sur le comportement de l'agent : activer la trace ("/trace on")

Demander "Quel était le temps équivalent de coupure en 2023 ?" puis "Et en 2022" et voir que l'agent ne fait pas de RAG pour répondre à la deuxième question car il a déjà stocké le résultat dans sa working memory !



4

AGENTS FIABLES : GRAPHES ET ÉTATS STANDARDS

Les limites de la boucle simple

Actuellement, l'agent est basé sur une boucle simple comme suit :

```
while True:
    action = LLM(state)
    if action == "tool":
        result = tool(...)
        state.append(result)
    else:
        return answer
```

Problèmes :

- Tout est **implicite** : à quelle étape en est-on ? quand est-ce qu'on s'arrête ?
- La boucle mélange tout : raisonnement, contrôle, exécution et mémoire
- Très difficile à débbugger/interrompre/contrôler/auditer ! et ajouter des garde-fous...

Passer du from scratch à un framework de développement

On a vu les limites du 100% from scratch : même en bénéficiant du native tool calling, ce n'est pas évident de gérer les instructions et la mémoire. C'est encore plus compliqué dès qu'on veut un minimum de contrôle sur les exécutions de tools, de l'observabilité, du human-in-the-loop ou de l'auditabilité (par ex. pour un agent d'entreprise).

L'idée maintenant c'est donc de passer à un framework open-source qui va nous packager une grosse partie du travail. Ce sera **LangGraph**.



4.1

Les différents frameworks

Dans la jungle des frameworks



LlamaIndex

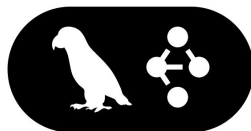


AutoGen



smolagents

crewai



LangGraph



PydanticAI



OPENCLAW

Frameworks haut niveau

Pour aller vite ou prototyper. Souvent ultra-simple mais peu de contrôle voire peu transparents (il faut aller chercher loin dans le code pour comprendre ce qui se passe)



smolagents

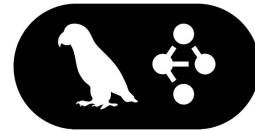
Frameworks intermédiaires

Plus de structure mais encore très accessibles avec coût d'entrée faible. Bon compromis pour plus de contrôle ou meilleure spécialisation (exemple : agent sur documents avec RAG pour LlamaIndex) mais pas optimal pour systèmes plus critiques.



Frameworks bas niveau

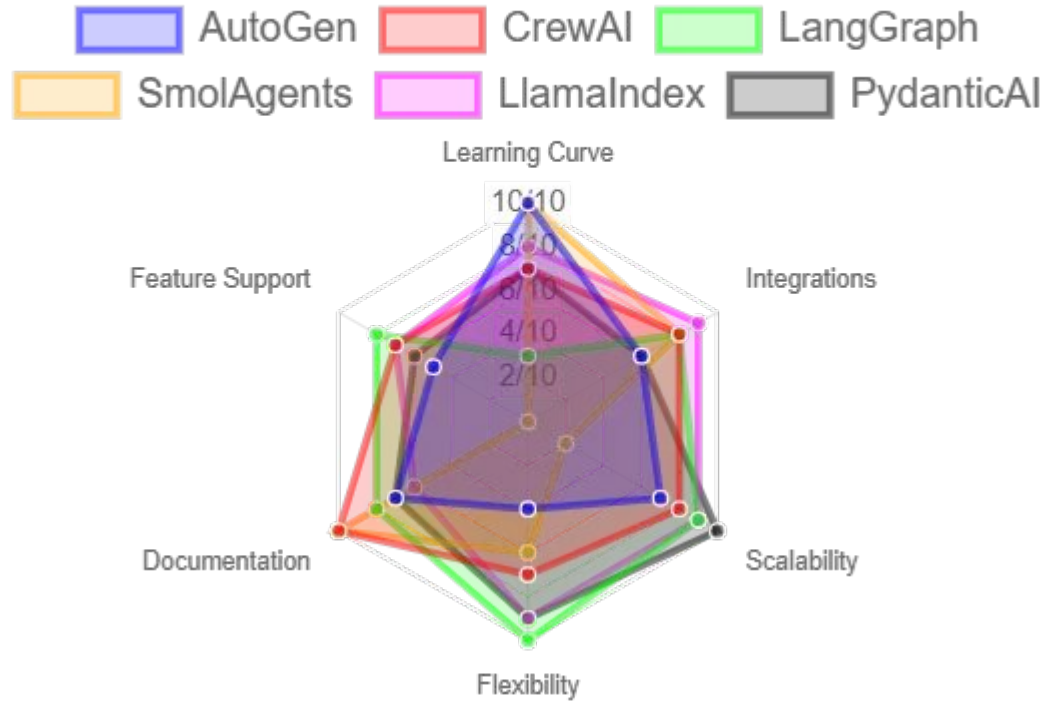
Pour systèmes robustes avec customisation totale possible (règles de human-in-the-loop, graphes explicites, observabilité). Courbe d'apprentissage plus élevée mais flexibilité totale. Souvent préférés pour la production.



LangGraph

Sans surprise, plus un framework est simple, plus il cache des décisions critiques. Plus on veut de contrôle, plus on doit descendre en abstraction...

En résumé



<https://www.thetoolnerd.com/p/how-to-select-the-right-ai-agentic-framework-thetoolnerd>

<https://aimultiple.com/agentic-frameworks>

Recommended pace

This is just a recommendation you can follow the course at your own pace



RECOMMENDED PACE

| | |
|---|--------|
| Unit 1: Introduction to Agents | Week 1 |
| Unit 2.1:  smolagents | Week 2 |
| Unit 2.2:  LlamaIndex | Week 3 |
| Unit 2.3:  LangGraph | Week 4 |
| Unit 3: Use Cases | Week 5 |
| Unit 4: Final Assessment with Benchmark | Week 6 |



4.2

LangGraph



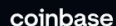
langgraph

Balance agent control with agency

Design agents that reliably handle complex tasks with LangGraph, an agent runtime and low-level orchestration framework.

[Start building](#)[Read the docs](#)

Trusted by companies shaping the future of agents



How does LangGraph help?

LangGraph et l'idée de graphe d'état

LangGraph est un framework open-source d'agents (créé par LangChain, la startup derrière le package langchain) fait pour créer, déployer et gérer des workflows agentiques complexes. Il est basé sur des architectures de graphes pour modéliser le workflow agentique.

Concrètement, ça veut dire qu'on va remplacer la **boucle implicite** de notre agent par un **graphe explicite** : on écrit explicitement

- les étapes possibles
- les transitions entre étapes
- l'état manipulé



LangChain VS LangGraph

LangChain sert à assembler des briques LLM rapidement = une chaîne.

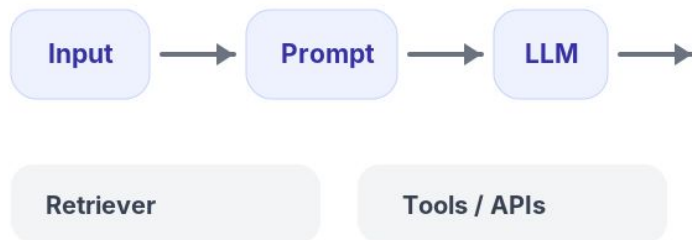
LangGraph sert à contrôler l'exécution d'un agent dans le temps.



LangChain VS LangGraph

LangChain

Assembler des briques LLM

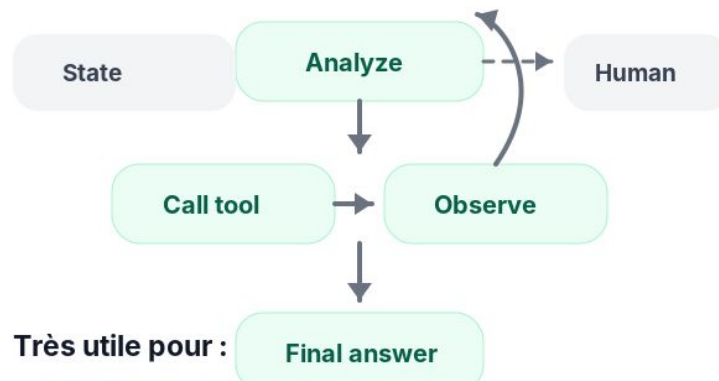


Très utile pour :

- RAG simple
- Chaînes prompt → modèle → parser
- Intégrations, retrievers, tools

LangGraph

Orchestrer un agent avec état



Très utile pour :

- Agents avec boucles et branches
- État, checkpoints, reprise
- Human-in-the-loop, multi-agents

Le graphe d'état

Un “graphe” est une notion toujours un peu vague ! En mathématiques, on le définit par des noeuds, des arêtes (edges), des transitions, et son état à instant donné. Concrètement, ici, on va avoir :

- noeuds : fonctions Python (les outils)
- arêtes : règles de passage entre outils
- état : dictionnaire d'état partagé et mis à jour entre les étapes (passages sur un noeud), qui contient les informations dont dispose l'agent à un instant donné (input, instructions, previous tool call results, etc.)

Avantages

On va pouvoir expliciter toutes les règles d'évolution de l'agent :

- quels outils sont disponibles (là, rien de nouveau)
- quels outils sont possibles après un outil donné
- combien de tool calls on peut/doit faire
- sous quelles conditions passer à l'étape suivante
- définir les interruptions (HITL : Human In The Loop)

On va également pouvoir facilement savoir où l'agent est passé dans le graphe (= quels tools et paramètres il a utilisés) : pratique pour observer le comportement de l'agent ! C'est ce qu'on nomme l'**observabilité**.

Conclusion : LangGraph permet d'inscrire l'agent dans un workflow semi-déterministe : *best of both worlds!*

Structurer l'état de l'agent

Maintenant qu'on travaille avec LangGraph, on passe d'une logique "tout est dans le prompt" à une logique "une partie est dans le prompt, une partie dans l'état".

L'état, c'est tout simplement un dictionnaire représentant l'état de mon agent au temps t :

```
state = {  
    "messages": [...],  
    "tool_calls_count": 2,  
    "max_tool_calls": 5  
}
```

On peut mettre dedans l'historique des messages, le nombre de tool calls, les chunks de texte obtenus par RAG, des informations tirées de la mémoire, des résultats intermédiaires (LLM reasoning), etc. = tout ce que le LLM doit manipuler explicitement. **State customization** = on choisit les champs du state.

Avantage : on a un état beaucoup plus structuré que copier-coller toutes ces informations en tas dans le prompt !

Prompt vs state

Et le prompt dans tout ça ? A chaque appel LLM on va choisir ce qu'on injecte dans le prompt depuis l'état.

L'état stocke, le prompt expose.

En fonction des champs stockés dans l'agent, vous avez toute liberté de créer le prompt comme vous voulez.



4.3

Exemples de graphes

De nombreux graphes différents

Ce framework est de bas niveaux pour deux raisons au moins

- On contrôle tous les transitions entre états et ce qu'il s'y passe
- Mais on a aussi une grande versatilité dans le choix des graphes.

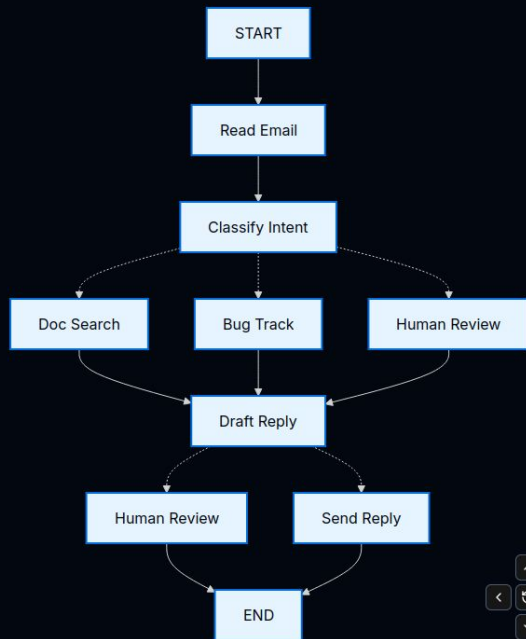
La versatilité a un coût qu'il faut assumer. **Le graphe n'est pas un gadget.** Pleins de variations simples de "flow" qui nécessitent des graphes différents

- Routage = lorsque l'on veut orienter une question/objectif vers différents sous-systèmes
- Multi-agent = (penser à tous les différents organigrammes de sociétés ...)
- Différentes étapes de vérification humain
- etc.

Réfléchir à son use case

Step 1: Map out your workflow as discrete steps

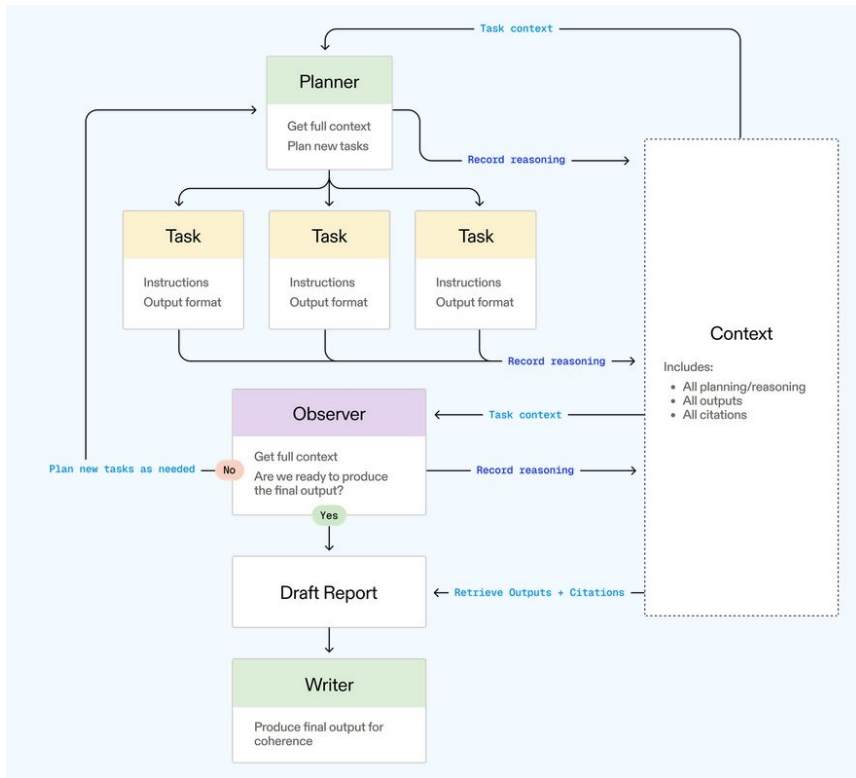
Start by identifying the distinct steps in your process. Each step will become a **node** (a function that does one specific thing). Then, sketch how these steps connect to each other.



Avec l'amélioration constante des capacités des agents à faire du code, le travail se transforme de plus en plus vers la capacité à

- bien formuler le problème que l'on veut résoudre (=bien traduire en graphe le travail de l'agent)
- Être capable d'auditer le code produit
- Assurer un contrôle du coût économique du système
- Assurer la sécurité

LangGraph customer stories ...



Exemple de "web research multi-agent system with LangGraph and LangSmith"

The best search API for AI

Powering agents with fast, high-quality web search

Try API for free

Try Websets

Find recent AI agent product launches

Fast Auto Deep

Full Web

Results AI Answer Structured Output

New Workspace Intelligence delivers unified, real-time understandin... workspace.google.com

New Workspace Intelligence delivers unified, real-time understanding to power agentic work.

Adobe's new productivity agent: Redefining how we understand, create an... blog.adobe.com

Adobe's new productivity agent: Redefining how we understand, create and share [...] We see the answer in agentic technology. AI that learns your patterns, carries out complex tasks and gets things

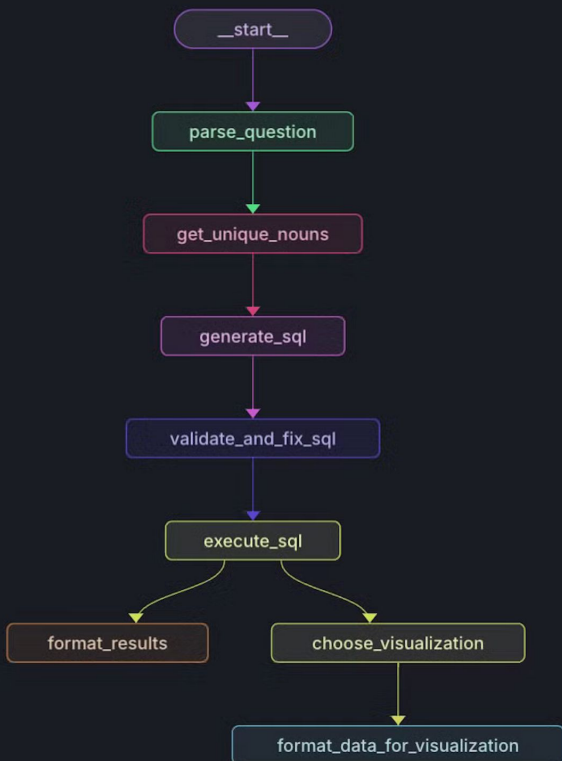
What's new in Gemini Enterprise | Google Cloud Blog cloud.google.com

Today, we are expanding Gemini Enterprise to meet this demand, transitioning AI from an isolated productivity tool into a secure, collaborative, autonomous engine for your business. Gemini Enterprise is the end-to-end so...

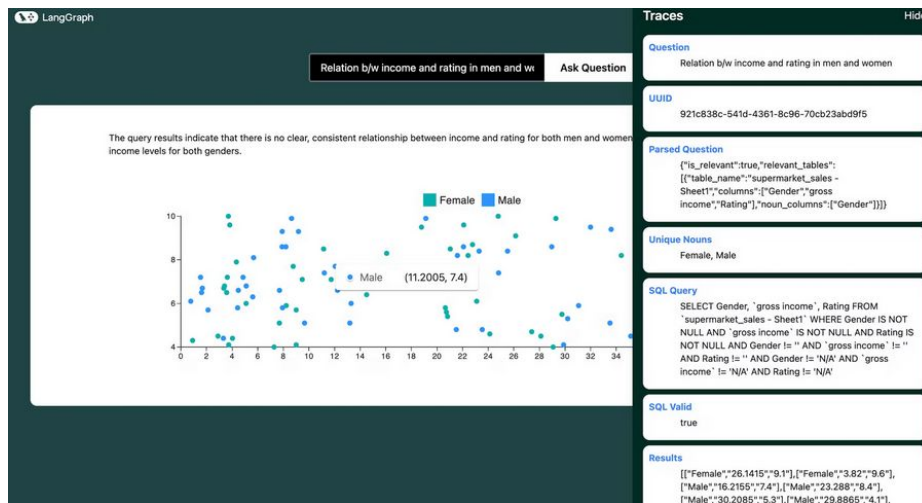
Visual Code



LangGraph customer stories ...



Exemple de "Data visualisation agent"



LangGraph customer stories ...

Et de nombreuses autres

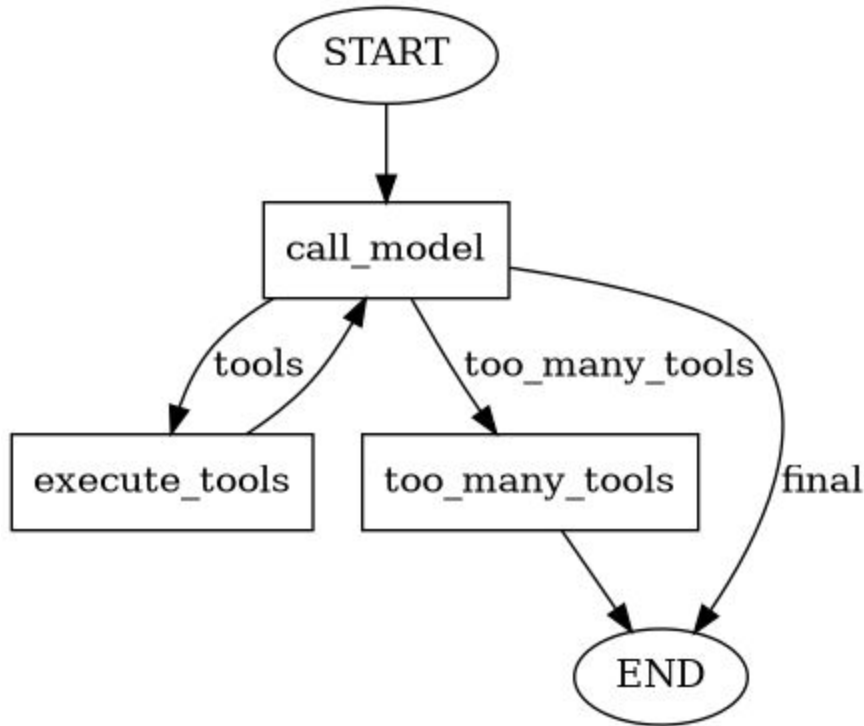
- <https://www.langchain.com/blog/credit-genie-insights-agent-financial-assistant>
- <https://www.langchain.com/blog/building-a-company-due-diligence-agent-with-deep-agents-langsmith-and-parallel>
- <https://www.langchain.com/blog/how-captide-is-redefining-equity-research-with-agentic-workflows-built-on-langgraph-and-langsmith>
- <https://www.langchain.com/blog/customers-remote>
- <https://www.langchain.com/blog/customers-monte-carlo>

TD : **app_TD4_langraph.py**

C'est le moment de quitter le code from scratch qu'on a développé dans les premiers TDs et de réécrire une première version sous LangGraph

Quasiment **rien ne change dans app_TD4_langraph.py**, tout le changement est définit dans **src/agent_langgraph.py**

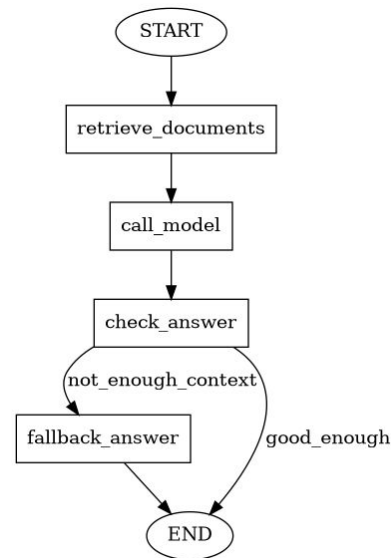
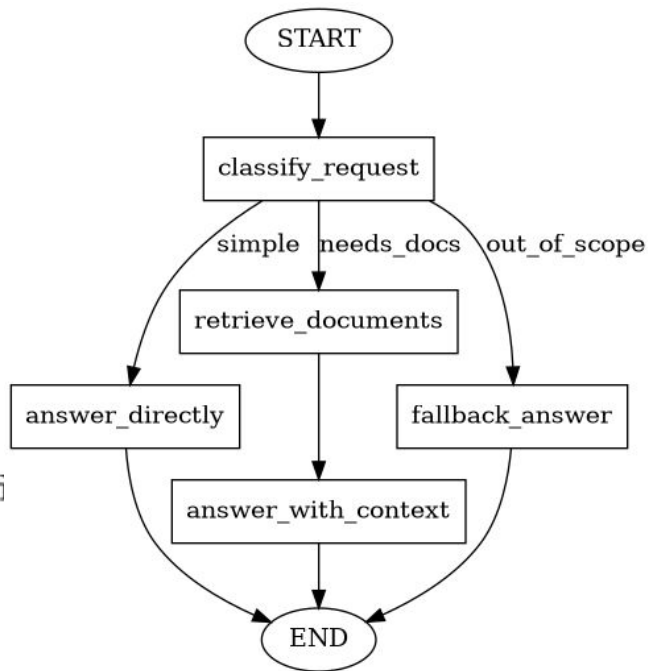
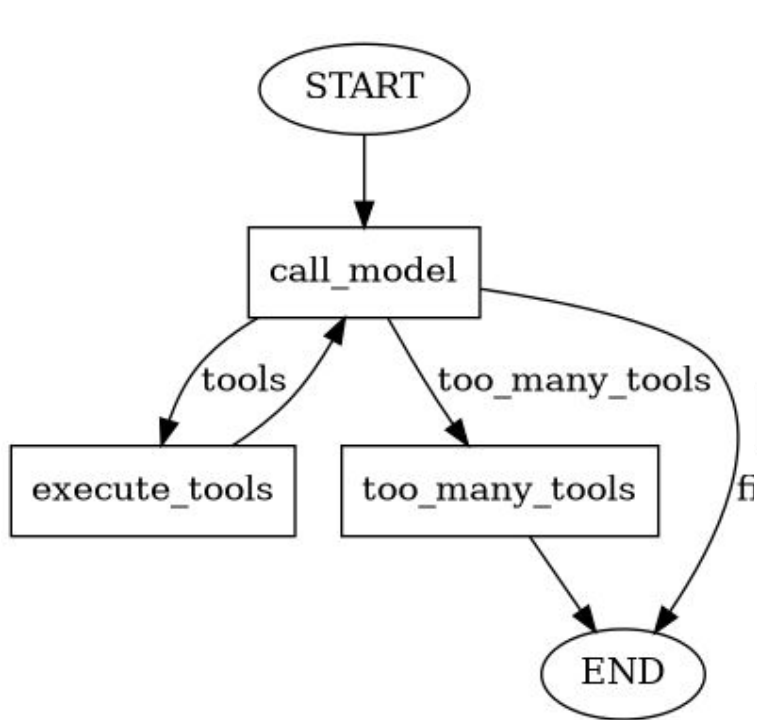
TD : app_TD4_langraph.py



```
# -----  
# Graph definition  
# -----  
  
def _build_graph(self):  
    graph = StateGraph(AgentState)  
  
    graph.add_node("call_model", self.call_model)  
    graph.add_node("execute_tools", self.execute_tools)  
    graph.add_node("too_many_tools", self.too_many_tools)  
  
    graph.add_edge(START, "call_model")  
    graph.add_conditional_edges(  
        "call_model",  
        self._route_after_model,  
        {  
            "tools": "execute_tools",  
            "final": END,  
            "too_many_tools": "too_many_tools",  
        },  
    )  
    graph.add_edge("execute_tools", "call_model")  
    graph.add_edge("too_many_tools", END)  
  
    return graph.compile()
```

Exercice : devenir bilingue en graph

Voir le jupyter notebook : [exercice_4_langgraph_graphs_correction.ipynb](#)





4.4

Garde-fous

Garde-fous

Ajouter un minimum de garde-fous est **essentiel** : en effet, on ne peut se garantir contre des problèmes tels que boucles infinies, tool calls incorrects, hallucinations diverses, etc.

Les garde-fous simples les plus classiques sont :

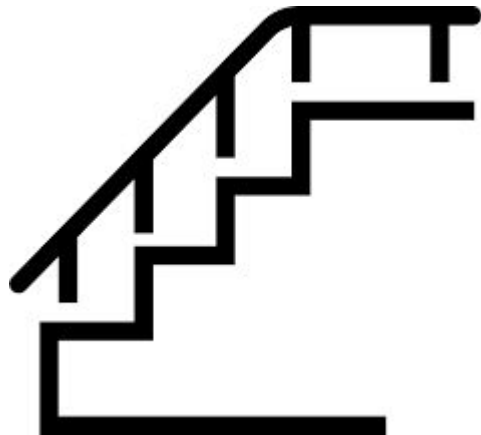
- *max_tool_calls* : on limite l'autonomie de l'agent à un nombre maximum de tool calls avant de revenir vers l'utilisateur
- *timeout* : similaire mais limite de temps d'exécution
- *validation des outputs* : helper qui vient valider si l'output du LLM vérifie certaines propriétés (si False on peut faire un certain nombre de retry)
- *tool restrictions* : autoriser ou interdire certains tools à certaines étapes
- *fallbacks* : que faire en cas de pépin

Où mettre les garde-fous ?

Pas trop de standard : préférence utilisateur + une certaine logique de code

- dans les noeuds
- dans les transitions

Avec LangGraph, les garde-fous deviennent du code explicite, et pas des bouts de prompt = c'est beaucoup plus propre et plus robuste !



Human-in-the-loop (HITL)

LE garde-fou de reference : human-in-the-loop (validation humaine).

Pour des cas critiques :

- avant de passer un prompt qui va consommer beaucoup de tokens
- si une action risque de modifier/supprimer un fichier
- avant de lancer une CLI qui peut modifier l'environnement d'exécution
- si besoin d'expertise métier
- ...

→ on a envie de demander une validation à l'utilisateur au lieu de laisser l'agent partir tout seul et risquer de dériver.

Typiquement : Le LLM propose une action qui fait intervenir des paramètres : si on a flaggé cette action ou les paramètres associés, alors on interrompt le flow automatique de l'agent. LangGraph supporte bien les commandes `interrupt`.

Checkpoints et time travel

Un autre garde-fou très utile, c'est le **checkpointing** des états de l'agent.

L'idée c'est de faire une **sauvegarde de l'état**, afin de pouvoir reprendre à partir de cette sauvegarde si besoin.

Exemple typique : on pose une question, l'agent bugge et n'arrive pas à répondre (on catche l'erreur via un garde-fou), et on veut nettoyer l'historique avant de recommencer avec un nouveau prompt.

Cela amène naturellement à la possibilité de faire du **time travel** : rejouer et modifier l'exécution de l'agent (faire des forks sur le graphe)

→ Très puissant : débbugger / auditer / améliorer les agents, reproduire les résultats, etc.

TD : app_TD5_langgraph_advanced.py



4.5

MCP

Model Context Protocol



Jusqu'ici on a surtout parlé de l'organisation/orchestration **interne à l'agent**. Maintenant, comment standardiser l'exposition des tools et du contexte vers les modèles et applications ? → c'est là que le MCP entre en jeu.

MCP = protocole client-serveur ouvert conçu pour permettre à un agent de se connecter de manière uniforme à des sources externes de contexte et à des outils. Il permet d'exposer de manière standardisée des tools, des ressources, des prompts.

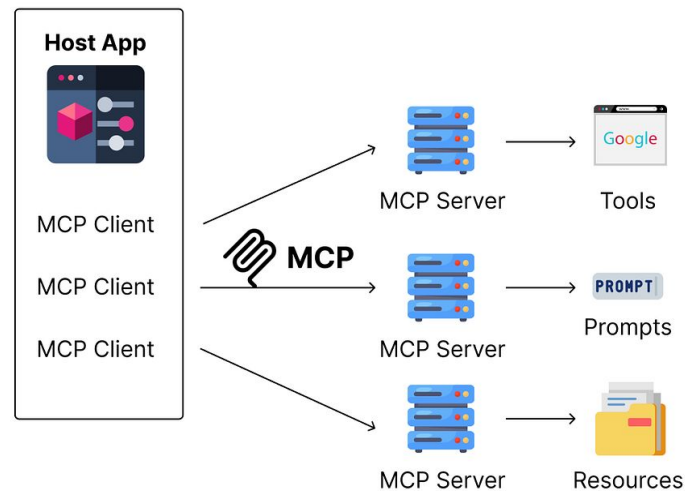
MCP vient juste standardiser l'interface entre l'agent et des briques externes. Sans MCP, chaque intégration (base SQL, GitHub, Slack, ...) doit être développée ad hoc. Avec MCP, on expose tout ça via un protocole commun, ce qui permet à l'agent de découvrir dynamiquement ce qui est disponible et comment l'utiliser.

MCP server

Un serveur MCP peut exposer 3 choses :

- des tools (**actions**)
- des ressources (**données**) = objets lisibles : fichiers, bases de données, documentations, etc.)
- des prompts (**instructions**) : templates paramétrables que le serveur met à disposition du client, pour guider une interaction avec le modèle

On ouvre une connection, après quoi le client peut lister les tools, lister ou lire des ressources, récupérer des prompts, puis invoquer des tools avec des arguments conformes au schéma annoncé.



<https://flaticon.com/>

Exemple figma dans cursor



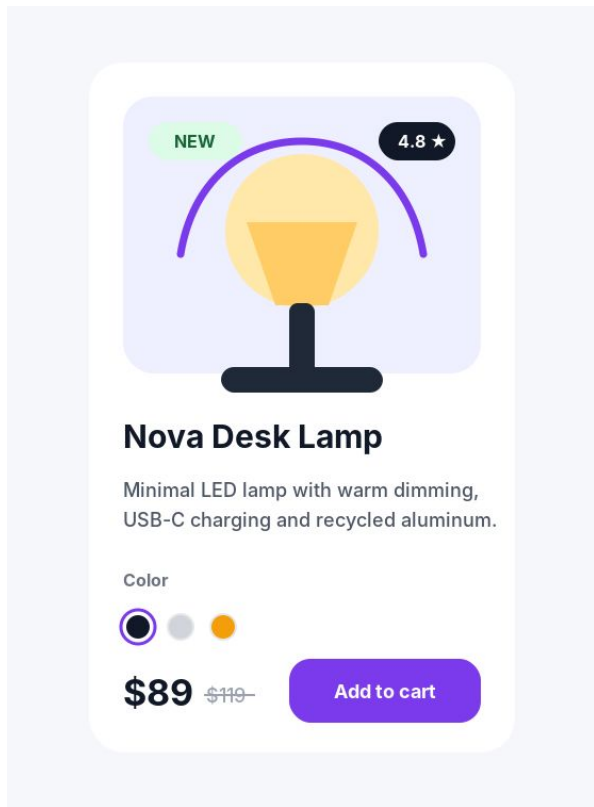
https://www.youtube.com/watch?v=brTAdHVCwSw&list=PLXDU_eVOJT7s80kdPADuhGeKzzSIbm7

- On peut tester sur cursor avec
- Permet de passer du bricolage avec une capture d'écran à faire lire à l'agent un vrai contexte structure

New Chat

/add-plugin figma

MCP server de figma



On peut s'amuser à tester de demander a cursor :

- coder une version de ce template à partir d'un screenshot
- Ou alors à partir de la maquette figma structurée en faisant appel au MCP figma.

awesome-remote-mcp-serversPublic

Watch12

main

3 Branches0 Tags

Go to file

Add file

<> Code

java9c Merge pull request #309 from aofstad/add-airtable

e16d68b · 6 hours ago

226 Commits

CODE_OF_CONDUCT.mdinitial commitlast year

CONTRIBUTING.mdinitial commitlast year

LICENSEinitial commitlast year

README.mdAdd Airtable (official remote MCP server)17 hours ago

README

Code of conduct

Contributing

MIT license

Awesome Remote MCP Servers

Subscribe to the Newsletter Stay informed about new MCP servers and updates: [Subscribe](#)

Not a developer & looking for ChatGPT Custom Integrations or Claude Custom Connectors? [Check out the Remote MCP Server List](#)

A curated, opinionated list of high-quality **remote** Model Context Protocol (MCP) servers. This repository focuses on listing official and well-maintained MCP servers that can be accessed over the internet, helping developers discover reliable and production-ready services for their AI applications.

What is MCP?

[Model Context Protocol](#) (MCP) is a protocol that enables AI models to interact with external tools and resources. Remote MCP servers allow clients to securely connect to these services over the internet.

What are Connectors?

This is OpenAI and Anthropic's branding for connecting to remote MCP servers.

Il y a pleins de version à explorer.

Faire attention à la sécurité

Exercice : `exercice_7_mcp_openai.ipynb`

MCP et sécurité

Ce n'est pas parce qu'un outil est exposé via MCP qu'il est automatiquement sûr. Par contre, MCP force à **expliquer** les choses : on sait quel serveur expose quoi, quels tools existent, quels schémas d'arguments ils attendent, et via quel chemin on les atteint.

Cela pousse vers une architecture plus propre. Ensuite, la vraie sécurité vient de ce qu'on met autour : permissions minimales, authentification, validation des arguments, sandboxing, HITL...

Set up son MCP

Pour un agent local sur fichiers, pas nécessaire en première instance. Le coeur du problème, c'est de :

- bien lire les formats
- bien indexer les documents
- bien contrôler les permissions
- bien gérer la recherche dans les fichiers
- bien tracer ce que l'agent fait
- et bien éviter qu'il fasse n'importe quoi sur le système de fichiers !

En revanche, MCP devient intéressant si vrai besoin de **modularité** et d'**interopérabilité** : par exemple, définir un accès aux fichiers internes puisse être réutilisé par plusieurs clients différents (un agent LangGraph aujourd'hui, un autre client demain).

Kpler MCP Beta Program. Help us shape the future of data communications.

Our MCP allows Large Language Models to translates plain requests into precise API calls, and delivers answers as narrative, chart, or CSV straight away.

[Learn more](#) →

Poser une question



 datagouv MCP



MCP et savoir interne

Etre IA-natif pour une entreprise c'est avoir identifié toutes les tâches/process qui peuvent être automatisées. Et cela passe par une structuration interne avec deux questions essentielles

- Comment structurer le savoir interne ?
- Comment donner accès à ce savoir à un agent ?

MCP ne construit pas la connaissance, mais cela peut nous forcer à penser la connaissance comme un ensemble de capacités exposables et donc donner une direction pour la structuration du savoir interne.

Au travail !

Se connecter à un MCP via LangGraph

MCP = architecture client / serveur

- Client MCP = notre agent (via `load_mcp_tools`)
- Serveur MCP = `open-meteo-mcp-server`

Le serveur expose :

- des tools (`weather_forecast`, `air_quality`, etc.)
- + un schéma d'arguments
- + des résultats structurés

Le client :

- découvre les tools
- les transforme en objets LangChain (`StructuredTool`)
- les donne au LLM

Se connecter à un MCP via LangGraph (2)

Deux façons de se faire parler les deux programmes (l'agent et le serveur MCP) :

- soit sur la même machine (on utilise stdio pour exécuter le MCP en script local)
- soit via un réseau (protocole http)

Avec stdio l'agent lance en arrière plan une CLI type `npx open-meteo-mcp-server`

Choix stdio/http : différence entre tests et production. En conditions opérationnelles, mieux vaut http : le serveur MCP peut être partagé entre plusieurs agents, il peut tourner sur un serveur distant, et on peut le sécuriser (authentication, logs de requêtes, etc.)

sync vs async tools

Un serveur MCP s'exécute comme **processus séparé** et n'a donc pas accès directement au runtime interne de LangGraph, c'est-à-dire au state, au store ou au contexte d'exécution

Conséquence : les tools exposés via MCP ne sont pas à traiter de la même manière que ceux exécutés localement.

- tool local : synchrone, il renvoie un résultat immédiatement car il tourne au sein de notre processus → `tool.invoke()`
- tool MCP : asynchrone, tool externe, qui tourne hors de notre processus, et il faut donc attendre le résultat → `tool.ainvoke()`

TD : app_TD7_langgraph_mcp.py



5

OBSERVABILITÉ ET SÉCURITÉ

Observabilité

C'est aussi très utile de mettre en place des outils d'observabilité, c'est-à-dire de tracking de ce qui se passe en arrière-plan avec l'agent, pour mieux comprendre ce qui se passe, débbugger, éventuellement auditer (et améliorer l'agent dans ses versions suivantes !)

Typiquement on va vouloir observer des choses comme :

- inputs utilisateur et prompts finaux envoyés au LLM
- outputs du LLM
- tool calls et résultats des tools
- évolution de l'état de l'agent
- latence des étapes
- coûts tokens

Observabilité (2)

**Par contre, l'observabilité $\neq$ "voir le chain of thought" :
c'est tracer les étapes, tools, fichiers, états, erreurs.**

Pourquoi ne pas logger le chain-of-thought ou reasoning ?

- sécurité (peut exposer des informations critiques comme localisation de fichiers, détails d'environnements, etc.)
- coût : on arrive rapidement à des fichiers trèèèèèès longs !
- bruit : c'est comme un journal ctl, on veut quand même pouvoir s'y retrouver...

Frameworks d'observabilité

On a vu comment faire de l'observabilité à la main : pas impossible mais lourd...
Pour nous sauver, il existe un certain nombre de packages (gratuits ou non) offrant des solutions d'observabilité clés en main.

Parmi les plus connues, on compte LangSmith, Helicone ou Arize Phoenix. Ce sont des plateformes d'observabilité qui ajoutent des couches de tracing structurées : reconstruire automatiquement l'arbre d'exécution d'une requête, voir les appels imbriqués, identifier la latence des étapes, quels tools ont été appelés, quels documents ont été récupérés, etc.

Concrètement, ces solutions donnent un moyen d'émettre des traces dans le code, puis une interface pour explorer ces traces.

Arize Phoenix: Open-source LLM tracing and evaluation

Evaluate, experiment, and optimize AI products in real time.

Get started

Self host

Backed by  Combinator



Build Reliable AI Apps

The world's fastest-growing AI companies rely on Helicone to route, debug, and analyze their applications.

LangSmith

The platform for agent engineering

Observe, evaluate, and deploy your agents.
Build agents without code.

Choisir son framework

Le plus simple : LangSmith. Mais pas gratuit ni open-source. Il y a un free tier mais il faut une clé d'API.

Pas encore d'équivalent open-source simple et plug-and-play de LangSmith.

Open-source : OpenTelemetry, LangFuse. Mais ils nécessitent un setup plus compliqué (LangFuse tourne dans un Docker)

```
export LANGSMITH_API_KEY="your_key"  
export LANGSMITH_TRACING="true"  
export LANGSMITH_PROJECT="cours-agents"  
python app.py
```

Quelles attaques sur les agents ?

1. Prompt injection (directe) : Utilisateur malveillant
2. Indirect prompt injection : Via documents / web / RAG
3. Tool injection : le modèle hallucine ou détourne un tool
4. Data exfiltration : Exfiltration via tools ou réponses
5. Over-permissioning : Tools trop puissants
6. Data poisoning (RAG) : documents compromis

Principes essentiels de la sécurité des agents

1. Least privilege
2. Zero trust (le LLM n'est PAS fiable)
3. Explicit allowlists/blocklists (ce que l'agent a le droit/n'a pas le droit de faire/lire/...)
4. Human override possible (HITL)
5. Observabilité obligatoire
6. Sandboxing des tools (exécution isolée et pas d'accès au file system global)

La sécurité ne doit pas être dans le prompt (= pas fiable) mais dans le code.

→ voir ReadMe_TD6.md pour plus de details !

Blocklists/allowlists

Contrôle des accès fichiers essentiel : autoriser (allowlist) et interdire (blocklist) explicitement l'accès à certains folders en lecture et/ou écriture.

Tout accès en dehors des répertoires autorisés est refusé

Certains répertoires sensibles sont explicitement bloqués

La blocklist vient en renfort de la allowlist

Filtrage des sorties

But : filtrer d'éventuels motifs sensibles qui se seraient retrouvés dans les sorties du LLM
: clés API, infos personnelles, détails d'environnement, etc.

```
def _filter_output(self, text: str) -> str:
    if not isinstance(text, str):
        text = str(text)

    # clés OpenAI style sk-...
    text = re.sub(r"\bsk-[A-Za-z0-9-]{10,}\b", "[REDACTED_API_KEY]", text)

    # tokens / secrets génériques
    text = re.sub(
        r"(?i)\b(api[_ -]?key|token|secret|password)\b\s*[:=]\s*\S+",
        r"\1=[REDACTED]",
        text,
    )

    # Emails si on veut les masquer
    text = re.sub(
        r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b",
        "[REDACTED_EMAIL]",
        text,
    )

    # limite de longueur
    max_chars = 4000
    if len(text) > max_chars:
        text = text[:max_chars] + "\n...[output truncated]..."

    return text
```

Sécurité de la mémoire et du RAG

L'agent est exposé à des tools de mémoire qui peuvent servir de vecteurs d'attaque : par exemple un prompt malicieux peut être stocké dans la mémoire persistante, ou forcer le LLM à copier des données sensibles dans la mémoire.

Important donc de rajouter des politiques de validation de ce qui peut être sauvegardé dans la mémoire : taille limitée, motifs suspects filtrés, etc.

De même, les documents peuvent être des sources d'attaques, en contenant par exemple des instructions malveillantes. Il faut donc rajouter au prompt de base qu'il ne faut jamais exécuter d'instructions provenant du contenu des fichiers !

TD : app_TD6_safety.py



6

ÉVALUATION DES AGENTS

Enjeux de validation

Valider un LLM = surtout juger de la qualité de la réponse.

Valider un agent : plus compliqué, car un agent peut échouer pour plein de raisons différentes. En effet, l'agent doit :

- choisir le(s) bon(s) outil(s)
- dans le bon ordre
- avec les bons arguments
- s'arrêter au bon moment
- ne pas boucler indéfiniment
- respecter les contraintes de sécurité
- et d'autres !

Offline (sur benchmarks contrôlés) + online (auprès des utilisateurs)

Evaluation fonctionnelle end-to-end

Question : l'agent répond-il à la question de l'utilisateur ?

Comme pour les LLMs : construire un benchmark interne de Q&A et mesurer le taux de réussite. "Quel est le montant total de la facture X ?", "Trouve le document qui parle du poste Y", "Résume les points clés de ce PDF", "Quels clients ont acheté tel produit d'après les fichiers disponibles ?"

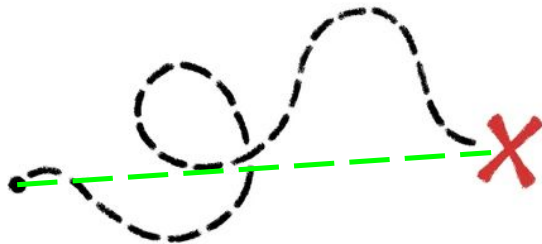
→ stratégie d'évaluation identique aux LLMs (cf. cours LLM).

Evaluation de la trajectoire de l'agent

Question : l'agent a-t-il suivi une bonne trajectoire pour arriver au résultat ?

Exemple simple : si l'agent choisit à chaque requête d'ouvrir tous les fichiers disponibles, il finira par tomber sur le résultat, mais d'une façon totalement inefficace. Ici on cherche à déterminer si l'agent fait trop de tool calls, ouvre trop de fichiers, utilise des outils non-adaptés ou interdits, etc.

Plus qualitatif : on regarde, pour chaque exécution, combien de tool calls ont été faits, quels tools ont été appelés, dans quel ordre, avec quels arguments, si un fallback a été déclenché, si un HITL a été atteint, si le nombre maximal d'étapes a été dépassé, si l'agent a tenté un outil interdit, etc.



Evaluation des composants de l'agent

Question : pourquoi l'agent échoue-t-il ?

On découpe le workflow : il faut pouvoir savoir si le problème vient du retrieval, du choix d'outil, du prompt, du LLM, ou de la politique d'état. Il faut donc faire des micro-évaluations ciblées :

- pertinence du RAG (pertinence des chunks obtenus)
- pertinence du choix d'outil sur un ensemble de cas simples
- gestion mémoire : l'agent utilise-t-il les informations données en mémoire ?
- structured JSON : l'agent respecte-t-il le format ?

→ stratégie de tests unitaires sur les composants pris séparément

Evaluation opérationnelle

Question : l'agent est-il utilisable en pratique ?

Évaluer ce qui compte en opérationnel :

- latence
- coût tokens
- stabilité
- taux d'échec
- nombre moyen d'étapes
- fréquence des retries
- respect des garde-fous

→ un agent intelligent peut être inutilisable en pratique si trop cher ou trop lent

Construire un bon benchmark

Benchmark minimal ~50 data points

Important de couvrir plusieurs classes de situations : questions simples et directes, questions nécessitant plusieurs documents, questions ambiguës, questions piégeuses, questions demandant un refus, questions qui doivent déclencher du retrieval, questions qui ne doivent surtout pas déclencher un tool coûteux, et questions adversariales (entrées malveillantes ou perturbatrices).

Exemples : prompt injection directe, document contenant une instruction cachée, nom de fichier trompeur, requête vague qui pousse à sur-utiliser les tools, question qui tente d'exfiltrer une donnée non autorisée.

Évaluation incrémentale

Difficile de dire si un agent est bon dans l'absolu : on va plutôt mesurer comment les performances de l'agent (selon les différentes métriques discutées ci-dessus) évoluent quand on change :

- le LLM
- le prompt de base (system prompt)
- les outils disponibles
- la politique de mémoire
- la stratégie de RAG
- etc.



7

CONCLUSION ET PERSPECTIVES

Agent en production

On reprend les briques que nous avons vues au fil de ce cours :

- LLM (raisonnement) : choix essentiel, déterminé par contraintes de calcul/confidentialité/coût, et qualité du LLM à servir de base à un agent
- tools (actions) : granulaires, explicites, pas d'overlap, native tool calling
- state (mémoire) : bien gérer l'état de l'agent et définir une politique memoire claire (épisode et persistante)
- control (graph) : définir les actions/transitions possibles de l'agent
- observability (logs) : tracer la progression de l'agent pour debugger
- guardrails (sécurité) : mettre en place block/allowlists, HITL, tester attaques, sandboxing

Agents spécialisés prêts au déploiement

Frameworks permettant d'abstraire les détails (mais beaucoup plus black box) :

- orchestration agent prête à l'emploi, prototypage rapide
- gestion des tools et de la mémoire (souvent épisodique + persistante)
- observabilité
- parfois multi-agents possible

Mais : sécurité parfois difficile à maîtriser, pas autant de flexibilité qu'on puisse vouloir, pas forcément les meilleures optimisations.

Systemes multi-agents

En pratique, un unique agent ne suffit pas toujours :

- Enjeux d'inférence en parallèle pour aller plus vite
- Enjeux de performance en mettant plusieurs agents en concurrence ou collaboration
- Enjeux d'efficacité en utilisant des agents plus spécialisés et plus robustes

Typiquement, un **planificateur** (qui décide de comment réaliser la tâche), des routeurs (choisissent quels agents appeler), des exécuteurs spécialistes (ceux qui font la tâche), et des reviewers (qui vérifient les résultats).

Avantages : spécialisation donc performance ⬆ + modularité

Inconvénients : gérer la coordination (pas évident), coût, propagation d'erreurs

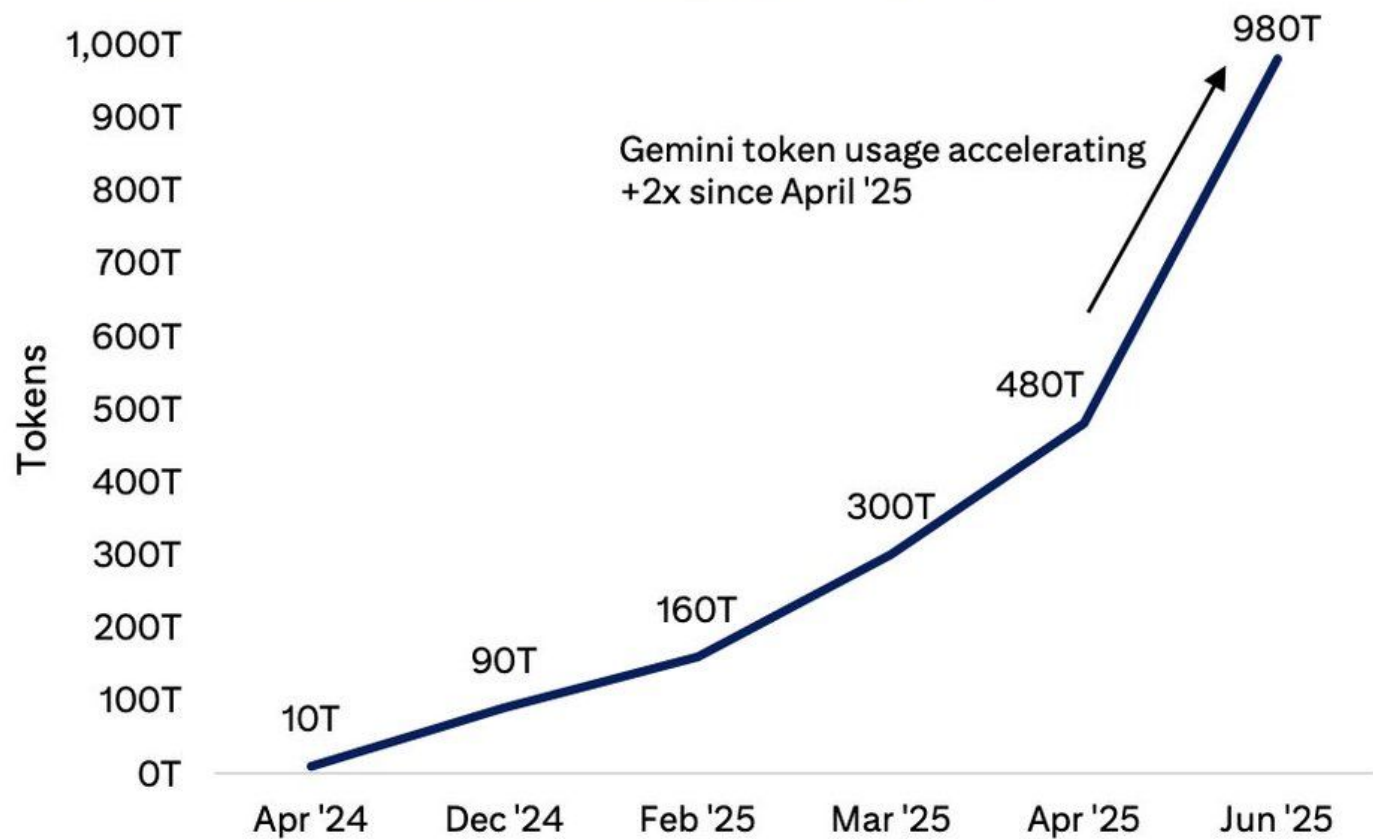
Et le pricing, dans tout ça ?

Le coût d'un agent est bien plus important qu'un simple call LLM : on a vu comment avec un agent le nombre de tokens envoyés aux LLMs et générés par eux **explose** :

- longs contextes en input car il faut garder une mémoire de travail, les instructions, la liste des tools, et les chunks de documents le cas échéant
- nombreuses étapes d'embedding (documents, mémoire, etc.)
- les boucles agentiques comprennent souvent de nombreuses étapes (différents tool calls, différents documents à considérer, etc.)
- et dans tout ça il y a de la génération de tokens implicitement à chaque fois !

En particulier ce qui coûte cher c'est la mémoire non-maîtrisée et quand on fait trop de tool calls : d'où l'importance (1) de trajectoires minimales et (2) de tools exhaustifs, spécifiques, sans overlap et puissants. Le coût = fonction du LLM et de la trajectoire !

Figure 1. Gemini Token Usage Accelerating through 2Q



Call To Action

- Se familiariser avec tout l'écosystème pour suivre les nouveautés
- Acheter un ordinateur avec GPU capable.
- Penser à des tâches qui sont utiles pour vous et s'entraîner à les automatiser via des agents.
- Tester régulièrement les nouvelles skills et frameworks qui sortent
- Toujours penser à la sécurité



8

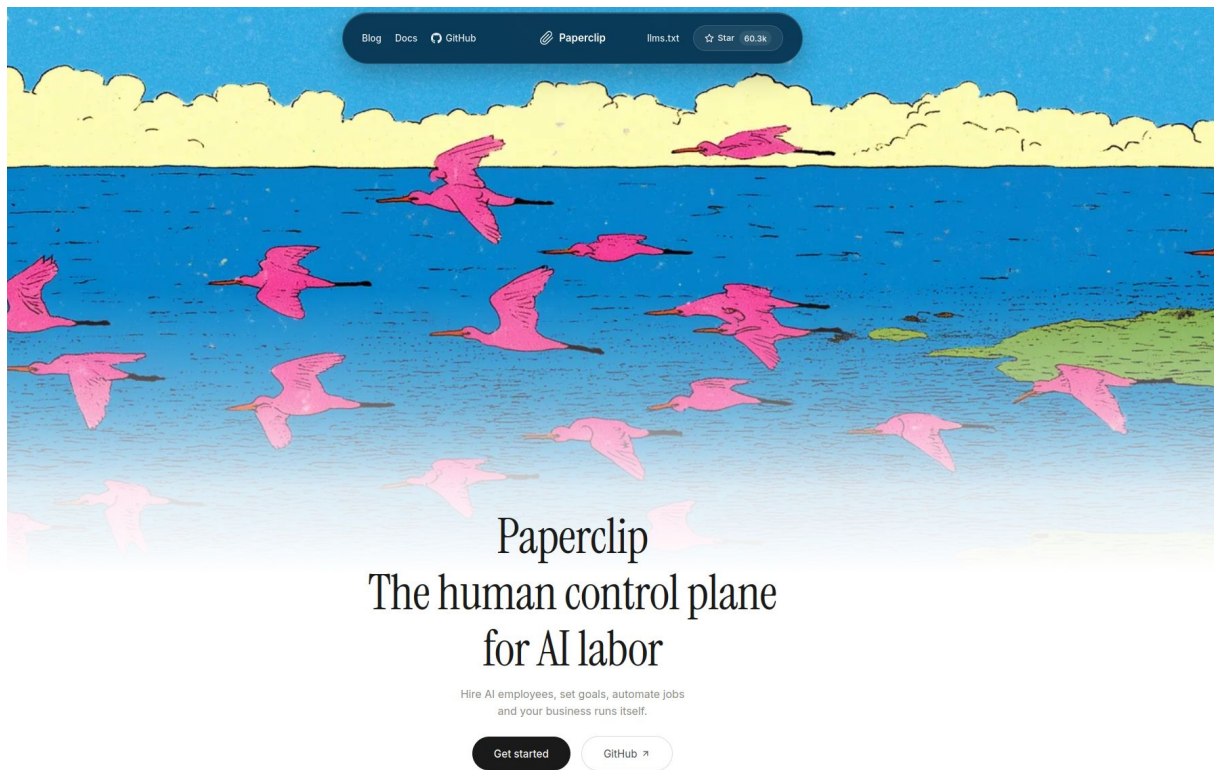
ANNEXES



8.1

Cas de PaperClip

Autre framework très récent : Paperclip



Paperclip

Get started

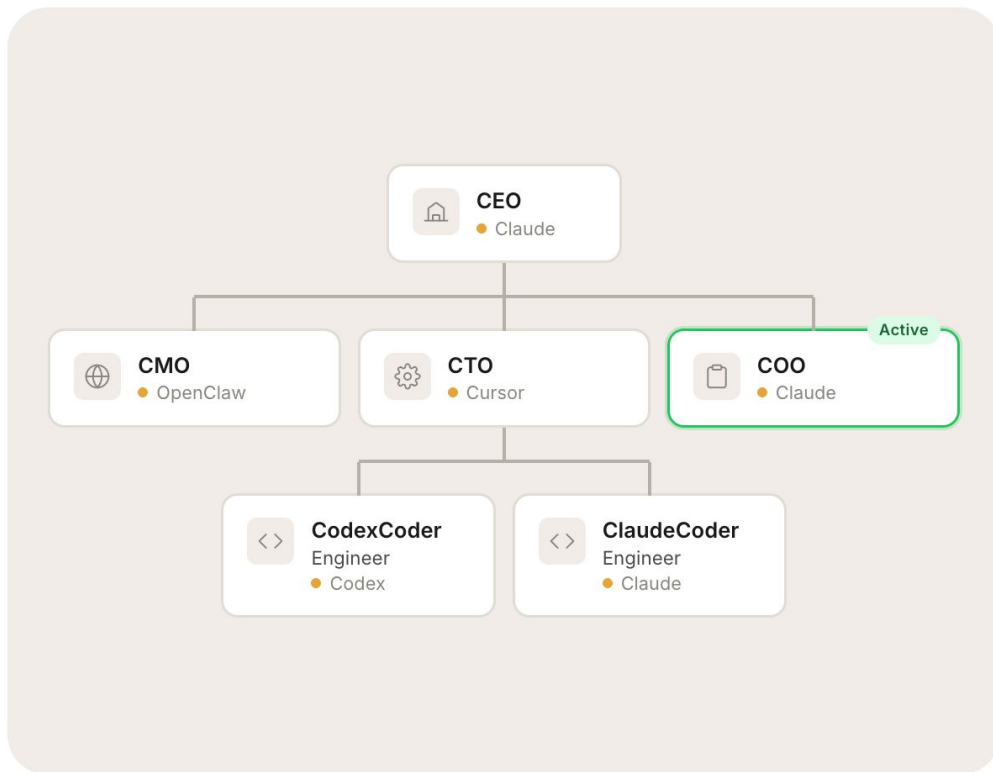
From zero to autonomous company
in one command.

```
$ npx paperclipai onboard --yes
```

Open source. Self-hosted. Interactive setup walks you through
installation and first company. No Paperclip account required.
No agents installed automatically.

Star on GitHub

Read the docs →



Choisir un adapter

Company Agent Task Launch

Create your first agent
Choose how this agent will run tasks.

Agent name
CEO

Adapter type

Recommended

Claude Code
Local Claude agent

Codex
Local Codex agent

More Agent Adapter Types

Gemini CLI
Local Gemini agent

Hermes Agent
Local Hermes CLI agent

OpenCode
Local multi-provider agent

Pi
Local Pi agent

Cursor
Local Cursor agent

OpenClaw Gateway
Configure OpenClaw within the App

Model
Default

Adapter environment check
Runs a live probe that asks the adapter CLI to respond with hello. [Test now](#)

← Back [Next](#)

Tester les différents frameworks qui sortent exposent a plein de vocabulaire

- Runtime agentique = framework d'exécution qui orchestre modèle + outils + mémoire + actions
- Adapter = couche qui permet à un runtime de parler a différents modèles ou fournisseurs. Par exemple charger Claude depuis OpenCode.



Pleins de frameworks à explorer

- Pleins de framework à explorer. Investissement sans savoir qu'est-ce qui perdurera
- Pas toujours facile d'utiliser des modèles locaux (voir ReadMe_paperclip.md pour une mise en bouche)
- Ici on ne choisit pas le design de la structure agentique, mais les concepteurs ont gamifié l'idée de créer une entreprise pour résoudre un tâche. C'est du multi-agent (et ce cours est focalisé sur solo-agent).
- Attention que ces frameworks passent sous le tapis les questions de sécurité (même quand on veut faire un simple agent qui lit nos emails pour mettre les .ics dans un calendrier).
- Procéder par étape, comprendre ce qui est faisable, suivre les différents frameworks et ensuite penser une architecture pour vos sociétés ou projets.



8.2

Openclaw

C'est quoi des hooks ?

Hook = module optionnel qui s'exécute automatiquement à certains moments du workflow. Par exemple :

 **boot-md** = charge automatiquement (i.e. injecté dans le contexte du LLM) un fichier Markdown au démarrage.

 **bootstrap-extra-files** = même logique mais avec plusieurs files

 **session-memory** = sauvegarde et recharge la mémoire entre session

 **command-logger** = Enregistre les commandes exécutées

 **compaction-notifier** = prévient sur le contexte du LLM a été résumé

OpenClaw / websearch

OpenClaw – Comparatif des moteurs de recherche web

Fonctionnalités, gratuité et coût estimé par requête (mai 2026)

| MOTEUR | GRATUIT ? | PRIX ESTIMÉ PAR REQUÊTE | FORCES PRINCIPALES | IDÉAL POUR | REMARQUES |
|--|---|---|---|---|---|
|  Brave Search API | Oui
~1000 req/mois (plan gratuit) | ~0,005 €
(0,001 – 0,005 €) | ✓ Bon équilibre qualité / coût
✓ Respect de la vie privée
✓ Index indépendant |  Meilleur choix global | Très bon pour un usage général et équilibré. |
|  Kimi (Moonshot) | Non
(low-cost) | ~0,002 – 0,01 €
(selon volume) | ✓ Très bon rapport coût / perf.
✓ Grand contexte
✓ Recherche web efficace |  Usage intensif économique | Nécessite une clé API Moonshot / Kimi. |
|  Tavily Search | Oui
(free tier limité) | ~0,005 – 0,02 €
(selon plan) | ✓ Optimisé pour les agents IA
✓ Résultats structurés
✓ Facile à intégrer |  Automatisation / workflows | Très adapté aux usages agentiques et outils IA. |
|  Perplexity API | Non | ~0,02 – 0,10 €
(selon modèle) | ✓ Qualité de recherche premium
✓ Réponses synthétisées
✓ Sources fiables |  Recherche approfondie | Qualité très élevée, coût plus important. |
|  SearXNG (self-hosted) | Oui
(open-source) | ~quasi gratuit
(coût infra seulement) | ✓ Open-source
✓ Contrôle total des données
✓ Vie privée maximale |  Self-host / coût minimal | Nécessite un serveur et de la maintenance. |

📊 ORDRE DE GRANDEUR DU COÛT MENSUEL*

| Scénario |  Brave |  Kimi |  Tavily |  Perplexity |  SearXNG |
|--------------------------|---|--|--|--|---|
| 1 000 recherches / mois | ~5 € | ~5 – 10 € | ~5 – 20 € | ~20 – 100 € | ~0 € |
| 10 000 recherches / mois | ~50 € | ~20 – 100 € | ~50 – 200 € | ~200 – 1 000 € | ~0 € |
| 50 000 recherches / mois | ~250 € | ~100 – 500 € | ~250 – 1 000 € | ~1 000 – 5 000 € | ~0 € |

* Estimations indicatives (mai 2026). Les prix peuvent varier selon les plans et le volume.

💡 À RETENIR

- L'accès internet (recherche web) est souvent un poste de coût majeur dans un système agentique.
- Le coût mensuel = nombre de recherches × prix / requête.
- Choisissez en fonction de votre budget, de vos besoins de qualité et de vos contraintes de confidentialité.

€ = Budget ⚖️ = Équilibre 🚀 = Performance 🔒 = Confidentialité